

UNIVERSIDAD DE GUANAJUATO



Programación en Ingenierías

Arreglos

Arreglo. Es un tipo de dato derivado, que representa un conjunto de elementos ordenados, donde todos los elementos son del mismo tipo. La cardinalidad (número de elementos en el arreglo), está definida y es conocida, además que los elementos en un arreglo están almacenados en espacios de memoria continuos.

Nota: El número de elementos en un arreglo debe estar definido como una constante, aunque las versiones superiores al estándar C99 permiten el definir el número de elementos en un arreglo como una variable.

Dimensión. La dimensión en un arreglo corresponde a las formas en que está estructurado un arreglo.

Índice. Es la posición de un elemento en una de las dimensiones de un arreglo, los índices deben ser enteros y el primer índice en un arreglo es el cero, por lo tanto el índice mayor siempre es la cardinalidad menos uno.

Sintaxis:

tipo identificador[N1][N2]...;

[][...]... El número de pares de paréntesis cuadrados corresponde a la dimensión.

N1, N2 Para cada dimensión se debe definir un número de elementos.

Tipos de arreglos.

Por el número de dimensiones:

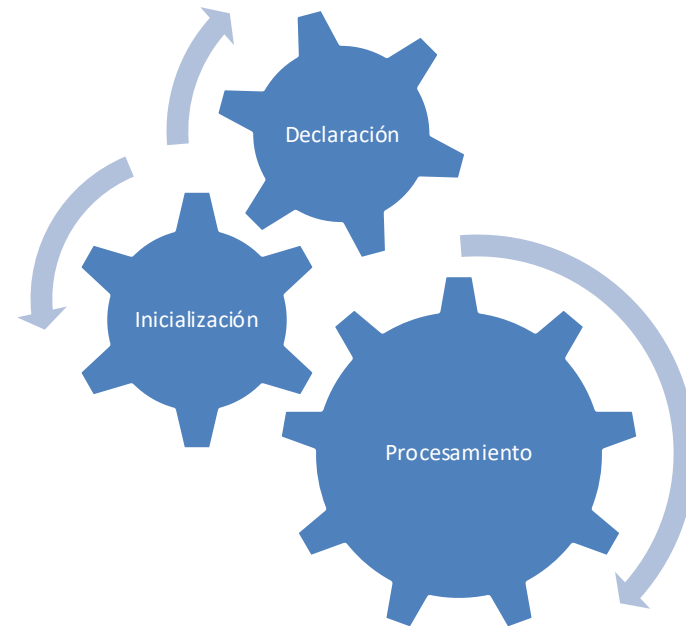
1. Unidimensional
2. Bidimensional
3. Multidimensional

Por el tipo de datos

1. Cadenas (char)
2. Enteros (int)
3. Punto flotante (float y double)

Etapas en el manejo de arreglos.

1. Declaración
2. Inicialización
3. Procesamiento



Arreglo unidimensionales.

tipo identificador[#elementos];

Etapas en el manejo de arreglos.

1. Declaración

```
#define N 10
```

```
int A[10]={[3]=2, [0]=5};
```

```
float X[N];
```

2. Inicialización

```
for (i=0; i<N;i++)
```

```
    X[i] = 0;
```

3. Procesamiento

```
for(i=0; i<N;i++)
```

```
    X[i]
```

i	A
0	5
1	0
2	0
3	2
4	0
5	0
6	0
7	0
8	0
9	0

i	x
0	12.3
1	5.6
2	-3.5
3	0.6
4	1.1
5	4.5
6	6.7
7	74.2
8	-1.3
9	-6.7

```
x[2] = -3.5;  
x[3] = 0.6;  
X[7] = 74.2;
```

Generación aleatoria de números.

- Librería

stdlib.h

time.h

- Funciones

rand()

srand()

time()

- Valores importantes

RAND_MAX

NULL

La función *rand()* genera un número pseudoaleatorio en el rango de $[0, \text{RAND_MAX}]$, debe ser utilizada para forma cíclica para llenar un vector. La generación de numero pseudoaleatorios depende de la inicialización de los mismo mediante la función *srand()*

rand() $[0, \text{RAND_MAX}]$

rand() % *max* $[0, \text{max}]$

rand() % (*max* - *min*) $[0, \text{max} - \text{min}]$

rand() % (*max* - *min*) + *min* $[\text{min}, \text{max}]$

rand() / *RAND_MAX* $\{0,1\}$

(1.0 * *rand()*) / *RAND_MAX* $[0,1]$

(*max* * *rand()*) / *RAND_MAX* $[0, \text{max}]$

(*max* - *min* * *rand()*) / *RAND_MAX* $[0, \text{max} - \text{min}]$

$$\frac{\text{max} - \text{min} * \text{rand}()}{\text{RAND_MAX}} + \text{min} \quad [\text{min}, \text{max}]$$

$$\max = 80 \quad \min = 18 \quad \max - \min = 62$$

rand()

rand()%(max - min) rand()%(max - min) + min

i	A
0	35789
1	3456
2	2893
3	83
4	5579
5	2357
6	3457
7	2975
8	37965
9	7789

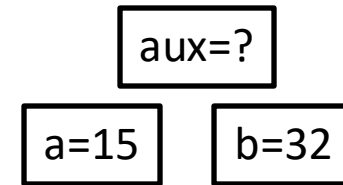
i	A
0	15
1	46
2	41
3	21
4	61
5	1
6	47
7	61
8	21
9	39

i	A
0	33
1	64
2	59
3	39
4	79
5	19
6	65
7	79
8	39
9	57

Algoritmo swap (Intercambio) – Variable auxiliar

Inicial

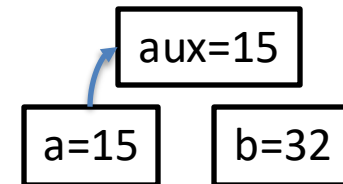
a = 15
b = 32
aux = ?



1

a = 15
b = 32
aux = 15

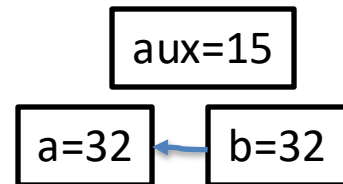
aux=a;



2

a = 32
b = 32
aux = 15

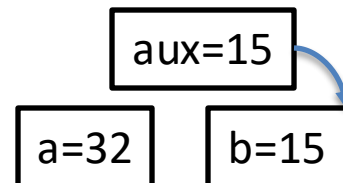
a=b;



3

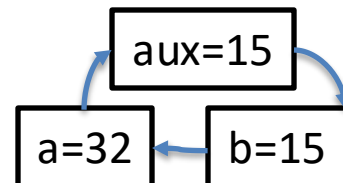
a = 32
b = 15
aux = 15

b=aux;



Final

a = 32
b = 15
aux = 15



Algoritmo swap (Intercambio) – Suma y resta

Inicial

 $a = 15$ $b = 32$ $a=15$ $b=32$

1

 $a = 15+32=47$ $b = 32$ $a+=b;$ $a=15+32=47$ $b=32$

2

 $a = 47$ $b = 47-32=15$ $b=a-b;$ $a=15+32=47$ $b=47-32=15$

3

 $a = 47-15=32$ $b = 15$ $a-=b;$ $a=15+32=47-15=32$ $b=47-32=15$

Final

 $a = 32$ $b = 15$ $a=32$ $b=15$

Algoritmo swap (Intercambio) – Multiplicación y división

Inicial

 $a = 15$
 $b = 32$ $a=15$ $b=32$

1

 $a = 15 * 32 = 480$
 $b = 32$ $a *= b;$ $a=15 * 32 = 480$ $b=32$

2

 $a = 480$
 $b = 480 / 32 = 15$ $b = a / b;$ $a=15 * 32 = 480$ $b=480 / 32 = 15$

3

 $a = 480 / 15 = 32$
 $b = 15$ $a /= b;$ $a=15 * 32 = 480 / 15 = 32$ $b=480 / 32 = 15$

Final

 $a = 32$
 $b = 15$ $a=32$ $b=15$

Operador binario XOR (^): El operador binario OR exclusiva implementa la operación binaria XOR, es verdaderos solo cuando son diferentes y es falso si son iguales.

$$A = 20_{10} = 00010100_2 \quad B = 15_{10} = 00001111_2$$

$$C = A \wedge B$$

<i>A</i>	00010100	20
<i>B</i>	00001111	15
<i>C</i>	00011011	27

$$D = A \wedge C$$

<i>A</i>	00010100	20
<i>C</i>	00011011	27
<i>D</i>	00001111	15

$$E = B \wedge C$$

<i>B</i>	00001111	15
<i>C</i>	00011011	27
<i>E</i>	00010100	20

A	B	A^B
0	0	0
0	1	1
1	0	1
1	1	0

Algoritmo swap (Intercambio) – Operador XOR

Inicial

 $a = 15$ $b = 32$ $a = 00001111 = 15$ $b = 00010000 = 32$

1

 $a = 15 \wedge 32 = 63$ $b = 32$ $a \wedge = b;$ $a = 00011111 = 63$ $b = 00010000 = 32$

2

 $a = 63$ $b = 63 \wedge 32 = 15$ $b \wedge = a;$ $a = 00011111 = 63$ $b = 00001111 = 15$

3

 $a = 63 \wedge 15 = 32$ $b = 15$ $a \wedge = b;$ $a = 00010000 = 32$ $b = 00001111 = 15$

Final

 $a = 32$ $b = 15$ $a = 32$ $b = 15$

Swap	Uso	Complejidad	Operador
Auxiliar	General	Alta	<code>aux=a;a=b;b=aux;</code>
Suma-resta	Enteros	Media-baja	<code>a+=b;b=a-b;a-=b;</code>
Multiplicación-división	Real	Media	<code>a*=b;b=a/b;a/=b;</code>
Operador XOR	Enteros	Menor	<code>a^=b;b^=a;a^=b;</code>

Análisis del desempeño de algoritmos.

La notación Big-O es en esencia una notación matemática, que se utiliza para comparar la tasa de convergencia de funciones. Sean $n \rightarrow f(n)$ y $n \rightarrow g(n)$ funciones definidas sobre los números naturales. Entonces decimos que $f = O(g)$ si y solo si $f(n)/g(n)$ está acotada cuando n tiende a infinito. En otras palabras, $f = O(g)$ si y solo si existe una constante A , tal que para todo n , $f(n)/g(n) \leftarrow A$.

El alcance de la notación Big-O es un poco más amplio en matemáticas, pero en el interés de nuestra temática, lo reduciremos a lo que se usa en el análisis de complejidad de algoritmos: funciones definidas en los naturales, que tienen valores distintos de cero, y el caso de n creciendo hasta el infinito.

Al comparar el rendimiento de un algoritmo, nos interesa la cantidad de operaciones que realiza un algoritmo. A esto se le llama complejidad temporal. En este modelo, consideramos que cada operación básica (suma, multiplicación, comparación, asignación, etc.) toma una cantidad fija de tiempo y contamos el número de tales operaciones. Por lo general, podemos expresar este número en función del tamaño de la entrada, que llamamos n .

Es necesario analizar el efecto del crecimiento de n al infinito (si no es así, decimos que el algoritmo es $O(1)$). Separamos nuestros algoritmos en clases de gran velocidad definidas por Big-O: cuando hablamos de un "algoritmo $O(n^2)$ ", queremos decir que el número de operaciones que realiza, expresado en función de n , es un $O(n^2)$. Esto dice que nuestro algoritmo es aproximadamente tan rápido como un algoritmo que haría un número de operaciones igual al cuadrado del tamaño de su entrada, o más rápido. La parte "o más rápido" está ahí porque utilicé Big-O en lugar de Big-Theta, pero por lo general la gente dice Big-O significa Big-Theta.

Ejemplos para analizar la complejidad en ciclos.

```
int max = INT_MIN;
for (size_t i = 0; i < len; i++) {
    if (max < array[i]) {
        max = array[i];
    }
}
```

El tamaño de entrada es el tamaño del vector, de la variable *len*.

Contando los operadores.

```
int max = INT_MIN;
size_t i = 0;
```

Estos dos operadores se realizan solo una vez, por lo que son 2 operaciones. Las operaciones que se enlazan son:

```
i < len
if(max < matriz [i])
i ++;
max = matriz [i];
```

$$O(n) = 4n + 2$$

Algoritmos de ordenamiento.

Los algoritmos de ordenamiento tiene como objetivo ordenar bajo un criterio específico un vector, permitiendo realizar un análisis más detallado del comportamiento de los datos.

Los algoritmos de ordenamiento son analizados mediante cinco criterios:

1. **Estabilidad:** Un algoritmo de ordenamiento es estable si conserva el orden relativo de los elementos iguales.
2. **En su lugar (memoria) :** Un algoritmo de ordenamiento está en su lugar si solo usa una sola memoria auxiliar (sin contar con una matriz auxiliar de ordenamiento).
3. **Complejidad del mejor caso:** Un algoritmo de ordenamiento tiene una complejidad $O(n)$ para el mejor de los casos considerado todas las posibles entradas.
4. **Complejidad del caso promedio:** Un algoritmo de ordenamiento tiene una complejidad $O(n)$ para el promedio de todos los posibles casos considerado todas las posibles entradas.
5. **Complejidad del peor caso:** Un algoritmo de ordenamiento tiene una complejidad $O(n)$ para el peor de los casos considerado todas las posibles entradas.

Algunos algoritmos de ordenamiento son:

- Algoritmo burbuja (Bubble sort)
- Algoritmo de pares y nones (Odd-Even sort)
- Algoritmo de selección (Selection sort)
- Algoritmo de inserción (Insertion sort)
- Algoritmo de casilleros (Bucket sort)
- Algoritmo burbuja bidireccional (Cocktail sort)
- Algoritmo Shell (Shell sort)
- Algoritmo de conteo (Counting sort)

Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	9
5	15
6	11
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posicionar que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posicionar que va ordenando.

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posicionar que va ordenando.

1. **Estabilidad:** Sí
2. **En su lugar (memoria) :** Sí
3. **Complejidad del mejor caso:** $O(n)$
4. **Complejidad del caso promedio:** $O(n^2)$
5. **Complejidad del peor caso:** $O(n^2)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

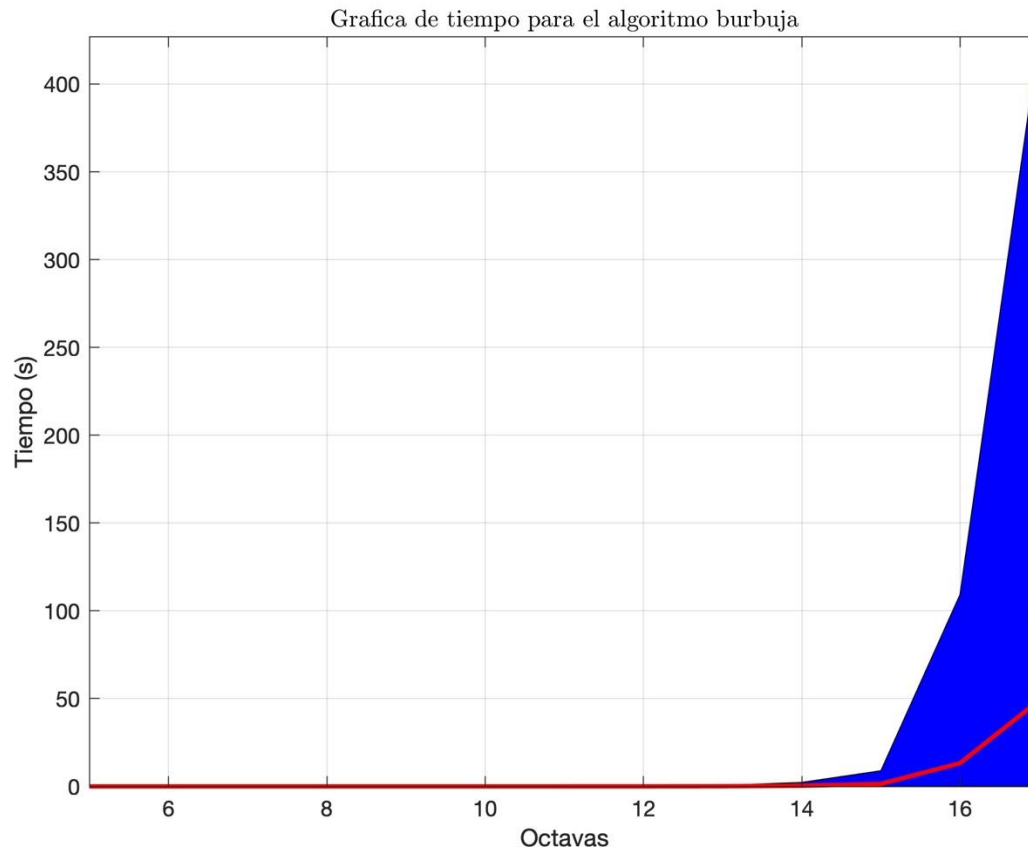
Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.



Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posiciones que va ordenando.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Características del sistema de cómputo:

Procesador: Intel Core i5 (4 núcleos)

1.4 GHz

Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo burbuja (Bubble sort)

El algoritmo burbuja es el método más simple de ordenamiento, también conocido como fuerza bruta, tiene un alto costo computacional, el método se basa en el principio de buscar el valor para cada una de las posicionar que va ordenando.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 10000

int main(int argc, char *argv[])
{
    int edad[N], i, j, n, max, min;
    float ep, med;
    do{
        printf("Ingrese el numero de personas: ");
        scanf("%d", &n);
    }while(n<1 || n>N);
    printf("Ingrese la edad maxima: ");
    scanf("%d", &max);
    printf("Ingrese la edad minima: ");
    scanf("%d", &min);
    if(max<min)
    {
        max^=min;
        min=max^min;
        max^=min;
    }
}
```

```
printf("[max = %d\tmin = %d]\n", max, min);
srand(time(NULL));
for(i=0; i<n; i++)
{
    edad[i]=rand()%(max-min)+min;
}
for(i=0; i<n; i++)
    printf("edad[%d] = %d\n", i+1, edad[i]);
for(i=1, ep=edad[0]; i<n; i++)
    ep+=edad[i];
ep/=n;
printf("Edad promedio: %f\n", ep);
for(i=0; i<n-1; i++)
    for(j=i+1; j<n; j++)
        if(edad[i]>edad[j])
        {
            edad[i]+=edad[j];
            edad[j]=edad[i]-edad[j];
            edad[i]-=edad[j];
        }
printf("Vector ordenado.\n");
for(i=0; i<n; i++)
    printf("edad[%d] = %d\n", i+1, edad[i]);
med = (n%2?edad[n/2]:(edad[n/2]+edad[n/2-1])/2.0);
printf("Mediana = %f\n", med);
return 0;
}
```

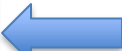
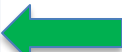
Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

I	X		I	X
0	5	←	0	3
1	3	←	1	5
2	8		2	8
3	9		3	9
4	15		4	15
5	9		5	9
6	11		6	11
7	16		7	16

Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

I	X		I	X
0	5		0	3
1	3		1	5
2	8		2	8
3	9		3	9
4	15		4	9
5	9		5	15
6	11		6	11
7	16		7	16

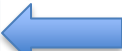
Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

I	X		I	X
0	3		0	3
1	5	←	1	5
2	8	←	2	8
3	9	←	3	9
4	9	←	4	9
5	15	←	5	11
6	11	←	6	15
7	16		7	16

Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

I	X		I	X
0	3		0	3
1	5		1	5
2	8		2	8
3	9		3	9
4	9		4	9
5	11		5	11
6	15		6	15
7	16		7	16

Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

I	X		I	X
0	3		0	3
1	5	←	1	5
2	8	←	2	8
3	9	←	3	9
4	9	←	4	9
5	11	←	5	11
6	15	←	6	15
7	16		7	16

Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

1. **Estabilidad:** Sí
2. **En su lugar (memoria) :** Sí
3. **Complejidad del mejor caso:** $O(n)$
4. **Complejidad del caso promedio:** $O(n^2)$
5. **Complejidad del peor caso:** $O(n^2)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

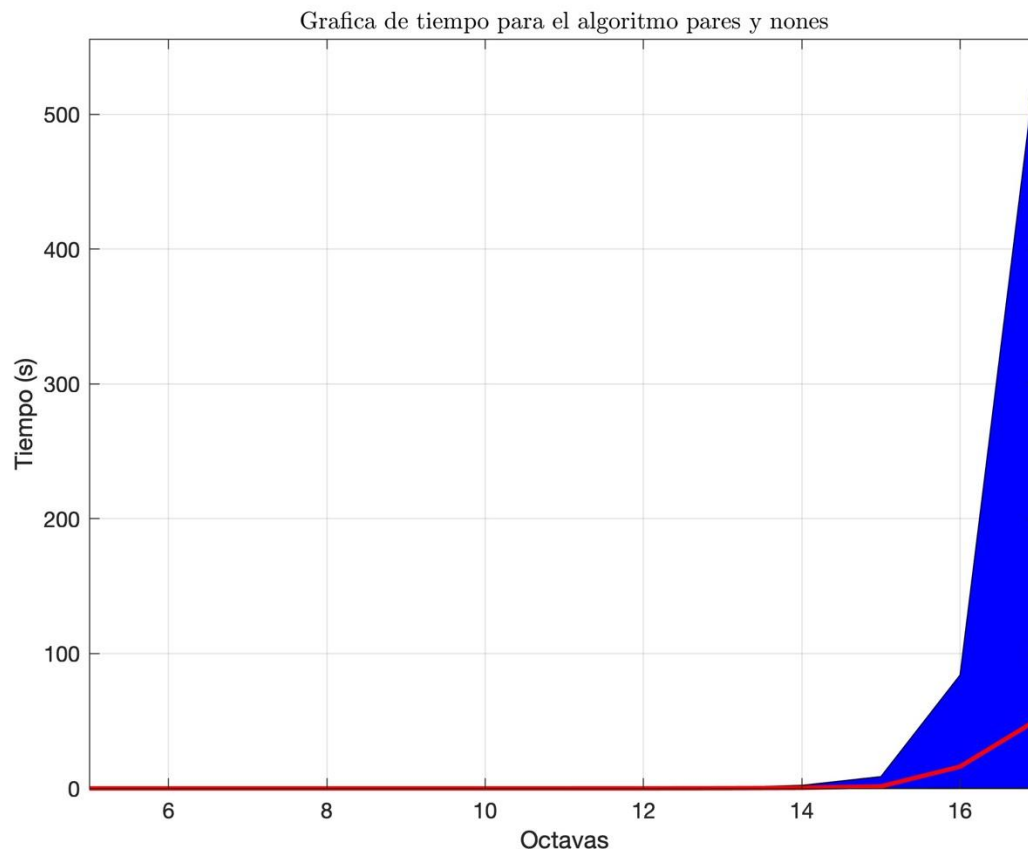
Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.



Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Características del sistema de computo:

Procesador: Intel Core i5 (4 núcleos)
1.4 GHz

Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de pares y nones (Odd-Even sort)

El algoritmo de pares y nones es un método estructurado piramidal basado en comparaciones y swap sucesivo que garantiza su ordenamiento.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

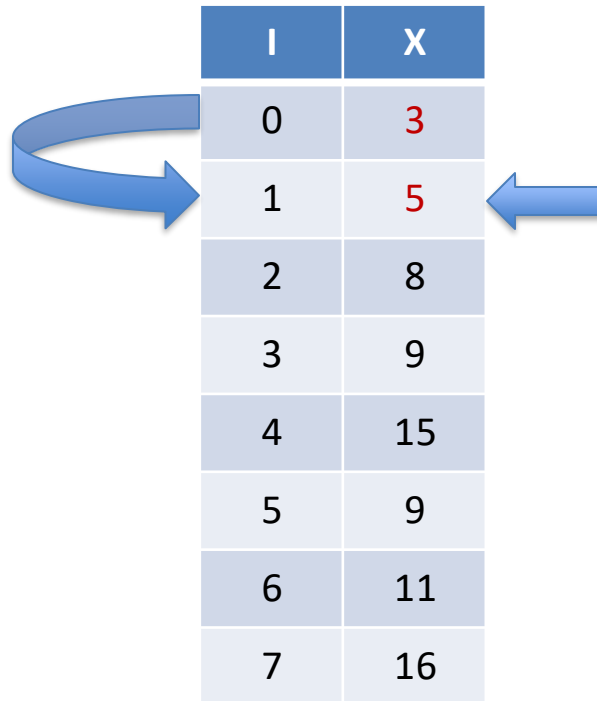
#define N 10000

int main(int argc, char *argv[])
{
    int edad[N], i, j, n, max, min;
    float ep, med;
    do{
        printf("Ingrese el numero de personas: ");
        scanf("%d", &n);
    }while(n<1 | n>N);
    printf("Ingrese la edad maxima: ");
    scanf("%d", &max);
    printf("Ingrese la edad minima: ");
    scanf("%d", &min);
    if(max<min)
    {
        max^=min;
        min=max^min;
        max^=min;
    }
    printf("[max = %d\tmin = %d]\n", max, min);
}
```

```
srand(time(NULL));
for(i=0; i<n; i++)
{
    edad[i]=rand()%(max-min)+min;
}
for(i=0; i<n; i++)
    printf("edad[%d] = %d\n", i+1, edad[i]);
for(i=1, ep=edad[0]; i<n; i++)
    ep+=edad[i];
ep/=n;
printf("Edad promedio: %f\n", ep);
for(i=0; i<n; i++)
    for(j=0; j<(n-i%2)/2; j++)
        if(edad[2*j+i%2]>edad[2*j+1+i%2])
        {
            edad[2*j+i%2]+=edad[2*j+1+i%2];
            edad[2*j+1+i%2]=edad[2*j+i%2]-edad[2*j+1+i%2];
            edad[2*j+i%2]-=edad[2*j+1+i%2];
        }
    printf("Vector ordenado.\n");
for(i=0; i<n; i++)
    printf("edad[%d] = %d\n", i+1, edad[i]);
med = (n%2?edad[n/2]:(edad[n/2]+edad[n/2-1])/2.0);
printf("Mediana = %f\n", med);
return 0;
```

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.



I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	←
Vector desordenado	3	9	
	4	15	
	5	9	
	6	11	
	7	16	

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
Vector desordenado	3	9	←
	4	15	
	5	9	
	6	11	
	7	16	

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
Vector desordenado	4	9	←
	5	15	
	6	11	
	7	16	

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
	4	9	
Vector desordenado	5	11	←
	6	15	
	7	16	

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
	4	9	
Vector desordenado	5	11	
	6	15	←
	7	16	←

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

Vector ordenado	I	X
	0	3
	1	5
	2	8
	3	9
	4	9
	5	11
	6	15
	7	16

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

1. **Estabilidad:** Sí
2. **En su lugar (memoria) :** Sí
3. **Complejidad del mejor caso:** $O(n)$
4. **Complejidad del caso promedio:** $O(n^2)$
5. **Complejidad del peor caso:** $O(n^2)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

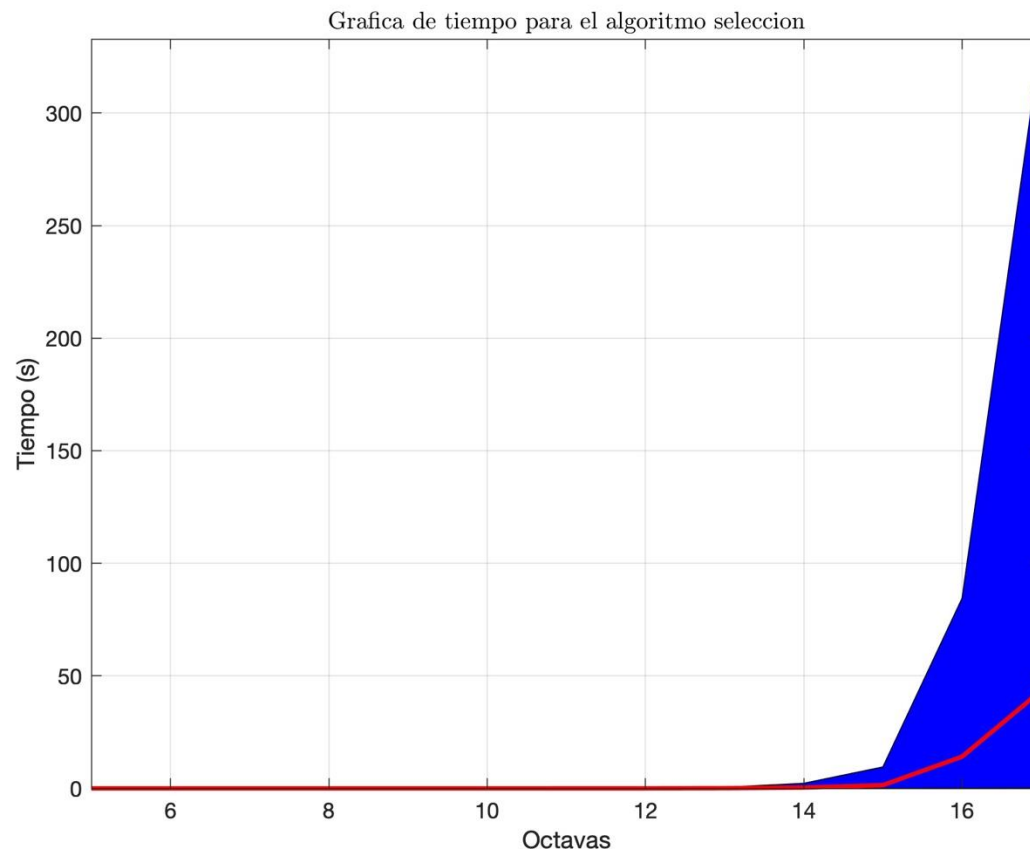
Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.



Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Características del sistema de cómputo:

Procesador: Intel Core i5 (4 núcleos)

1.4 GHz

Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de selección (Selection sort)

El algoritmo de selección divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector se localiza el mínimo o máximo según el orden, este elemento es eliminado de la segunda parte y lo se agrega al primer vector, el vector ordenando.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 10000

int main(int argc, char *argv[])
{
    int edad[N], i, j, n, max, min, imin;
    float ep, med;
    do{
        printf("Ingrese el numero de personas: ");
        scanf("%d", &n);
    }while(n<1 || n>N);
    printf("Ingrese la edad maxima: ");
    scanf("%d", &max);
    printf("Ingrese la edad minima: ");
    scanf("%d", &min);
    if(max<min)
    {
        max^=min;
        min=max^min;
        max^=min;
    }
    printf("[max = %d\tmin = %d]\n", max, min);
    srand(time(NULL));
```

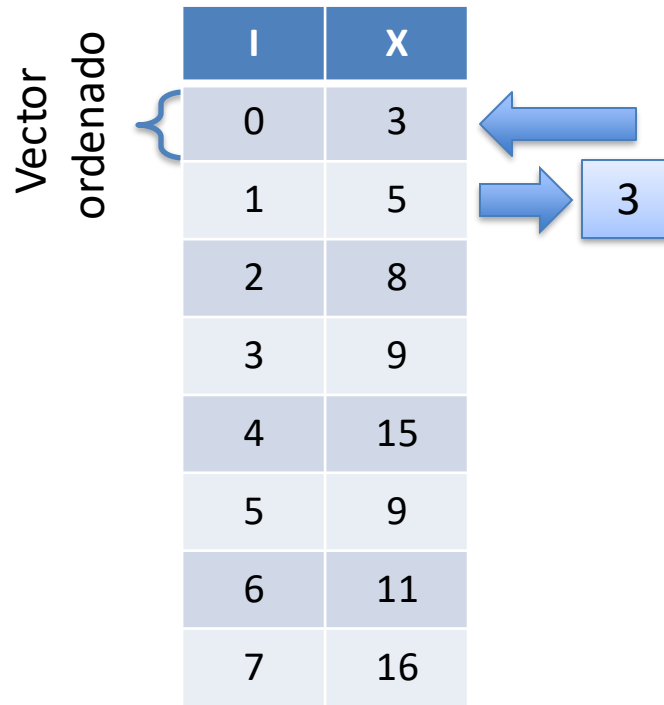
```
    for(i=0; i<n; i++)
    {
        edad[i]=rand()%(max-min)+min;
    }
    for(i=0; i<n; i++)
        printf("edad[%d] = %d\n", i+1, edad[i]);
    for(i=1, ep=edad[0]; i<n; i++)
        ep+=edad[i];
    ep/=n;
    printf("Edad promedio: %f\n", ep);
    for(j=0; j<n-1; j++)
    {
        for(i=j+1, imin=j; i<n; i++)
            if(edad[imin]>edad[i])
                imin=i;

        if(imin!=j)
        {
            edad[imin]^=edad[j];
            edad[j]^=edad[imin];
            edad[imin]^=edad[j];
        }
    }
    printf("Vector ordenado.\n");
    for(i=0; i<n; i++)
        printf("edad[%d] = %d\n", i+1, edad[i]);
    med = (n%2?edad[n/2]:(edad[n/2]+edad[n/2-1])/2.0);
    printf("Mediana = %f\n", med);
    return 0;
```

}

Algoritmo de inserción (Insertion sort)

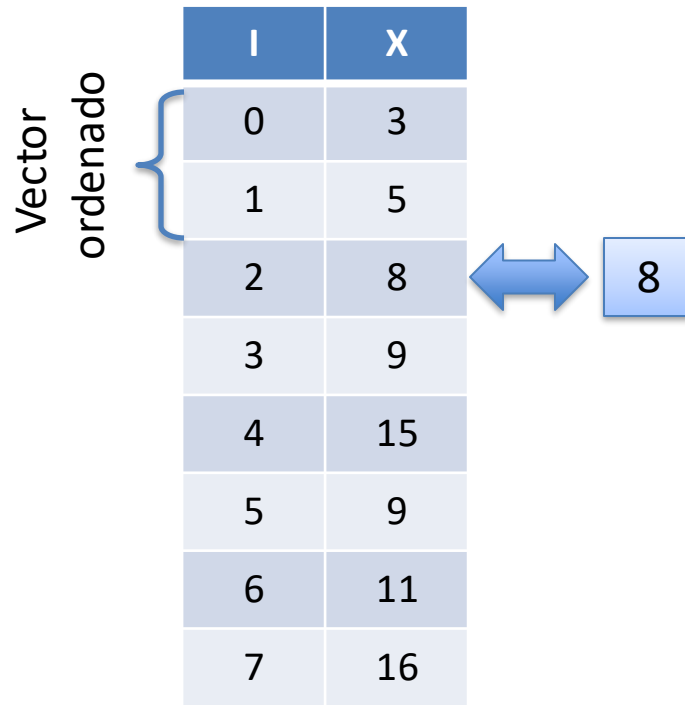
El algoritmo de inserción divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.



Vector ordenado	I	X
	0	3
	1	5
	2	8
	3	9
	4	15
	5	9
	6	11
	7	16

Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.



	I	X
Vector ordenado	0	3
	1	5
	2	8
	3	9
	4	15
	5	9
	6	11
	7	16

8

Algoritmo de inserción (Insertion sort)

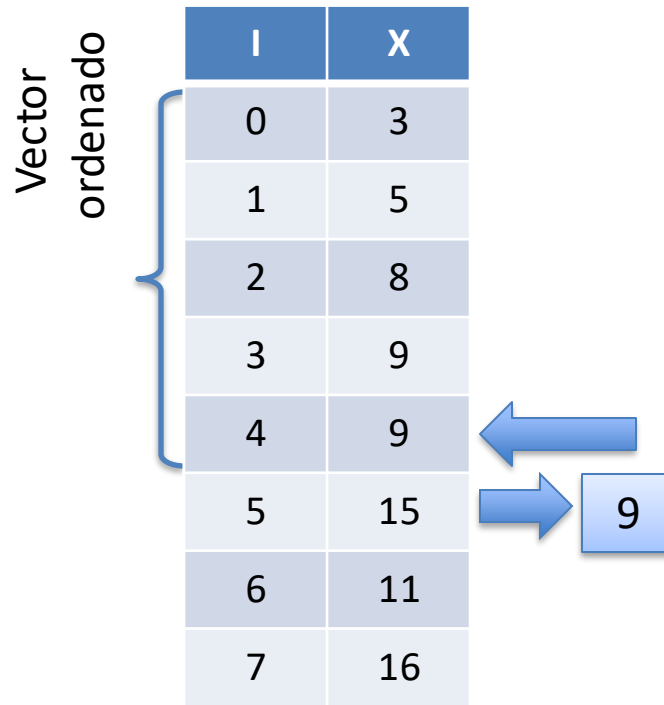
El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.

Vector ordenado	I	X
	0	3
	1	5
	2	8
	3	9
	4	15
	5	9
	6	11
	7	16



Algoritmo de inserción (Insertion sort)

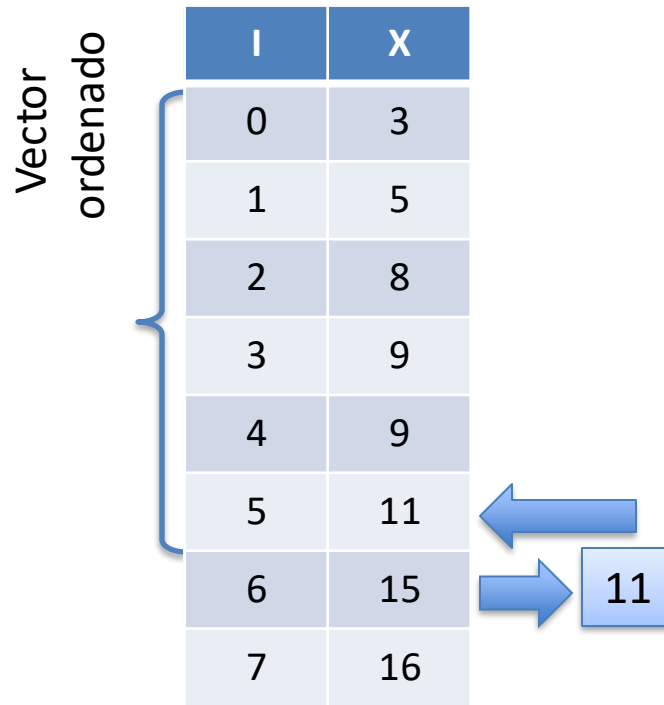
El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.



	I	X
Vector ordenado	0	3
	1	5
	2	8
	3	9
	4	9
	5	15
	6	11
	7	16

Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.



	I	X
Vector ordenado	0	3
	1	5
	2	8
	3	9
	4	9
	5	11
	6	15
	7	16

Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
	4	9	
	5	11	
	6	15	
	7	16	
		 16	

Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.

Vector ordenado	I	X
	0	3
	1	5
	2	8
	3	9
	4	9
	5	11
	6	15
	7	16

Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.

1. **Estabilidad:** Sí
2. **En su lugar (memoria) :** Sí
3. **Complejidad del mejor caso:** $O(n)$
4. **Complejidad del caso promedio:** $O(n^2)$
5. **Complejidad del peor caso:** $O(n^2)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

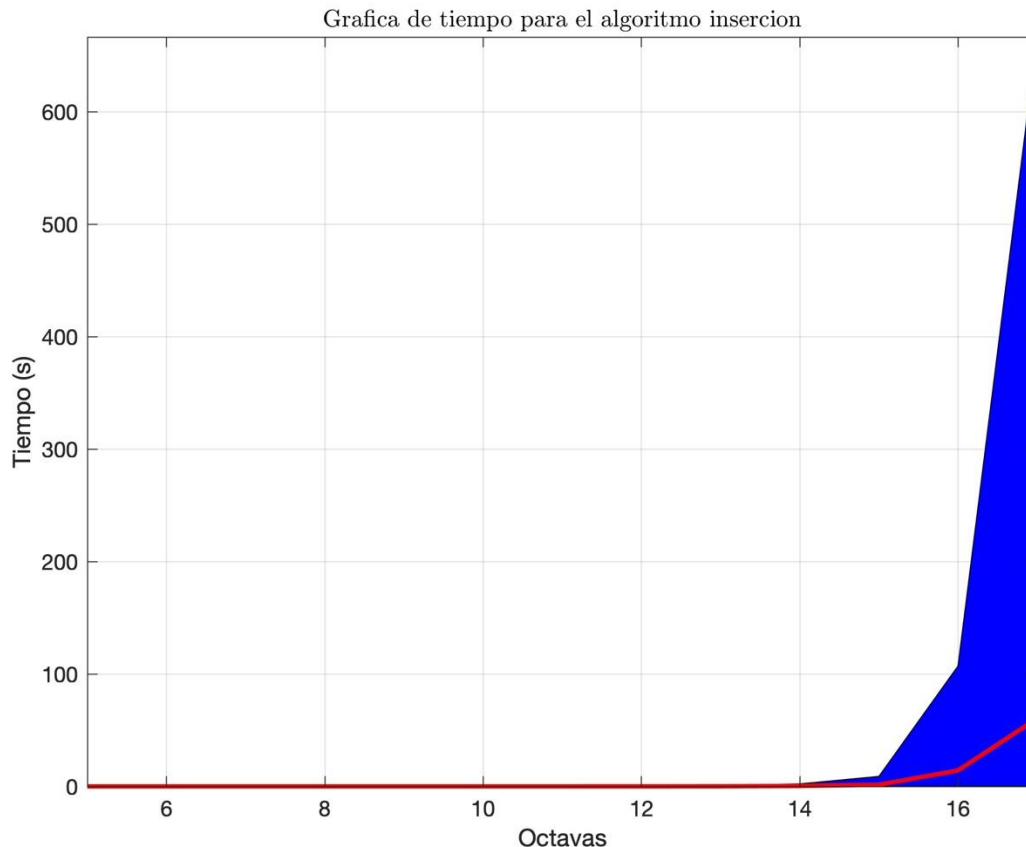
Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.



Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primera parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Características del sistema de cómputo:

Procesador: Intel Core i5 (4 núcleos)

1.4 GHz

Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de inserción (Insertion sort)

El algoritmo de inserción divide el vector en dos partes, la primer parte corresponde al vector ordenado, la segunda parte es el vector desordenado, del segundo vector (desordenado) se selecciona al primer elemento y se inserta en el primer vector (ordenado) según la posición que le corresponda.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 10000

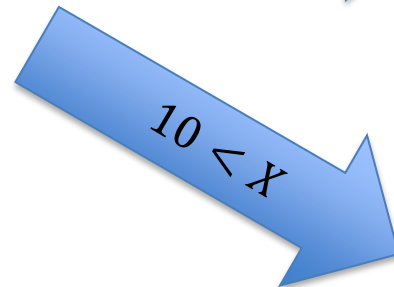
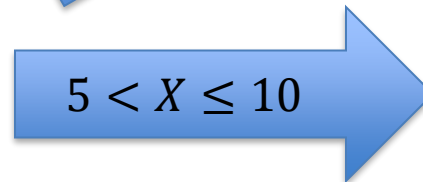
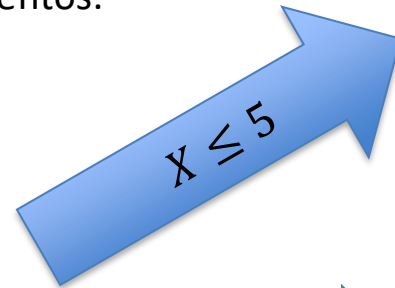
int main(int argc, char *argv[])
{
    int edad[N], i, j, n, max, min, aux;
    float ep, med;
    do{
        printf("Ingrese el numero de personas: ");
        scanf("%d", &n);
    }while(n<1 || n>N);
    printf("Ingrese la edad maxima: ");
    scanf("%d", &max);
    printf("Ingrese la edad minima: ");
    scanf("%d", &min);
    if(max<min)
    {
        max^=min;
        min=max^min;
        max^=min;
    }
    printf("[max = %d\tmin = %d]\n", max, min);
    srand(time(NULL));
```

```
    for(i=0; i<n; i++)
    {
        edad[i]=rand()%(max-min)+min;
    }
    for(i=0; i<n; i++)
        printf("edad[%d] = %d\n", i+1, edad[i]);
    for(i=1, ep=edad[0]; i<n; i++)
        ep+=edad[i];
    ep/=n;
    printf("Edad promedio: %f\n", ep);
    for(i=1; i<n; i++)
    {
        aux=edad[i];
        j=i-1;
        while(aux<edad[j])
        {
            edad[j+1]=edad[j];
            j--;
            if(j<0)
                break;
        }
        edad[j+1]=aux;
    }
    printf("Vector ordenado.\n");
    for(i=0; i<n; i++)
        printf("edad[%d] = %d\n", i+1, edad[i]);
    med = (n%2?edad[n/2]:(edad[n/2]+edad[n/2-1])/2.0);
    printf("Mediana = %f\n", med);
    return 0;
```

Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.

I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16



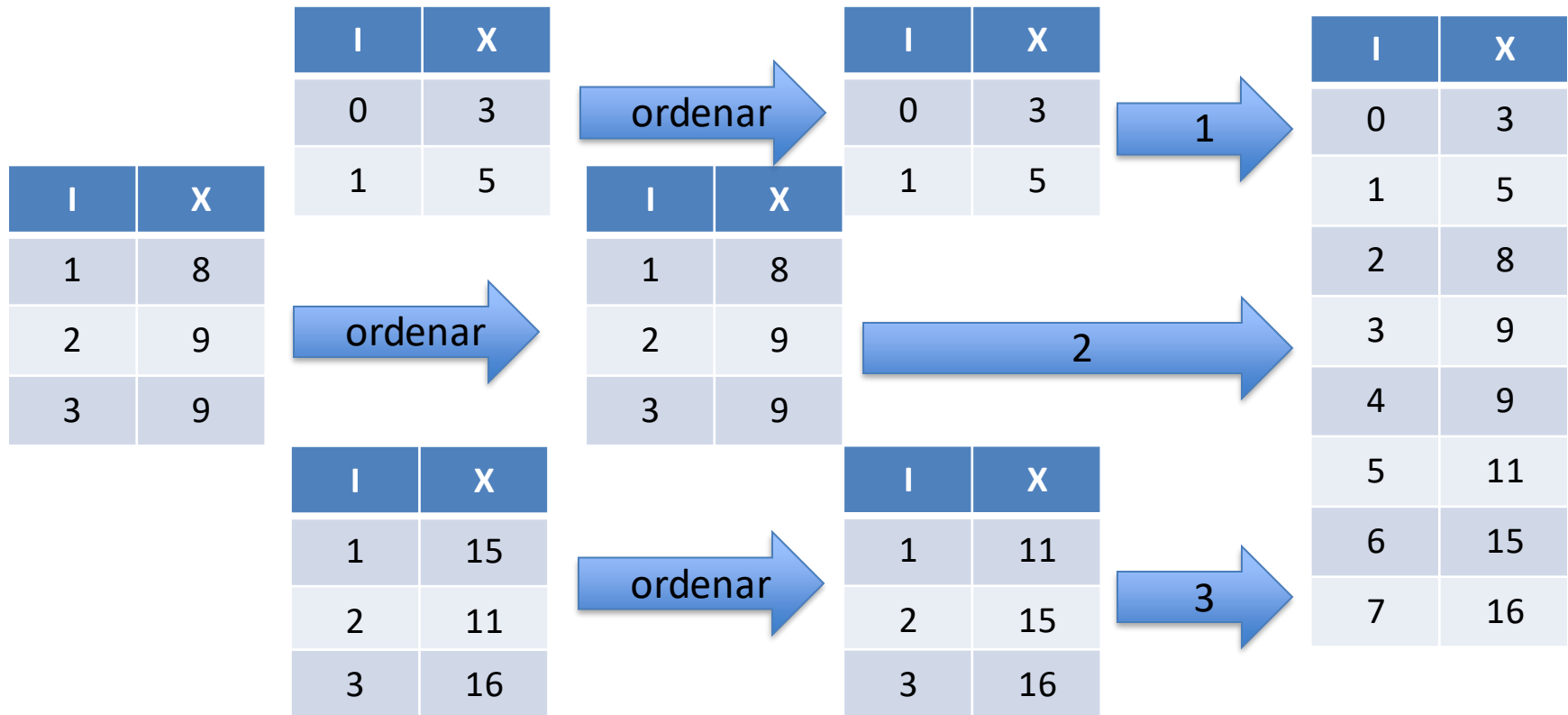
I	X
0	3
1	5

I	X
1	8
2	9
3	9

I	X
1	15
2	11
3	16

Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.

1. **Estabilidad:** Sí
2. **En su lugar (memoria) :** Sí
3. **Complejidad del mejor caso:** $O(n)$
4. **Complejidad del caso promedio:** $O(n)$
5. **Complejidad del peor caso:** $O(n^2)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

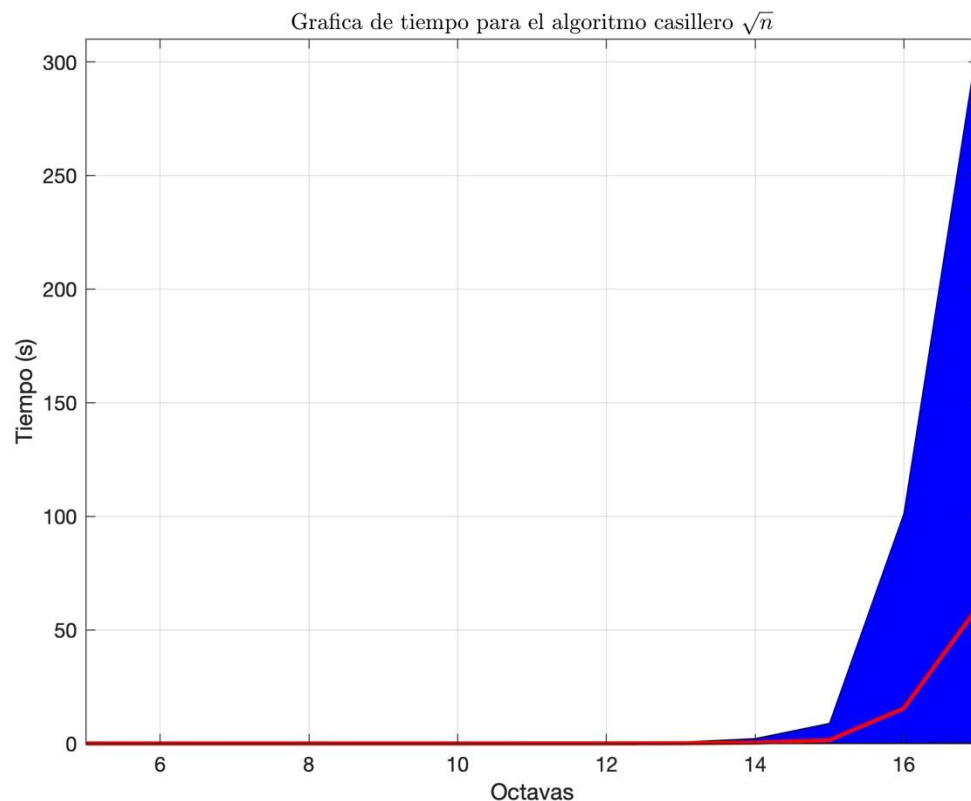
Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Considerando $\sqrt{n}$ casillas

Características del sistema de computo:

Procesador: Intel Core i5 (4 núcleos)
1.4 GHz

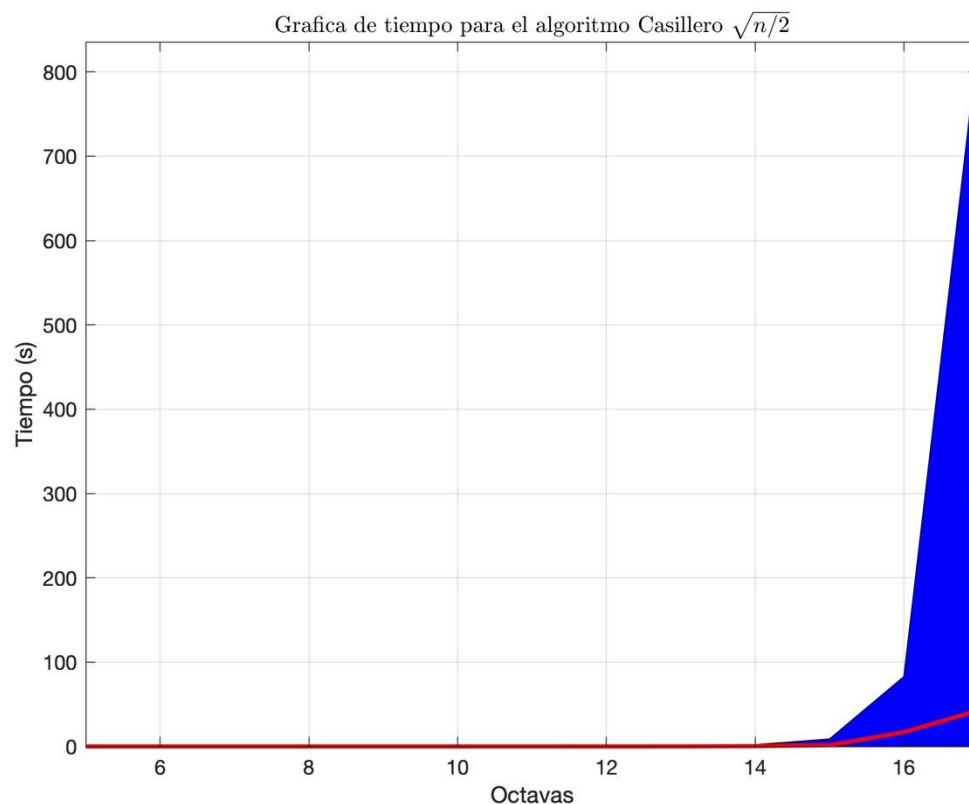
Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Considerando $\sqrt{n}/2$ casillas

Características del sistema de computo:

Procesador: Intel Core i5 (4 núcleos)

1.4 GHz

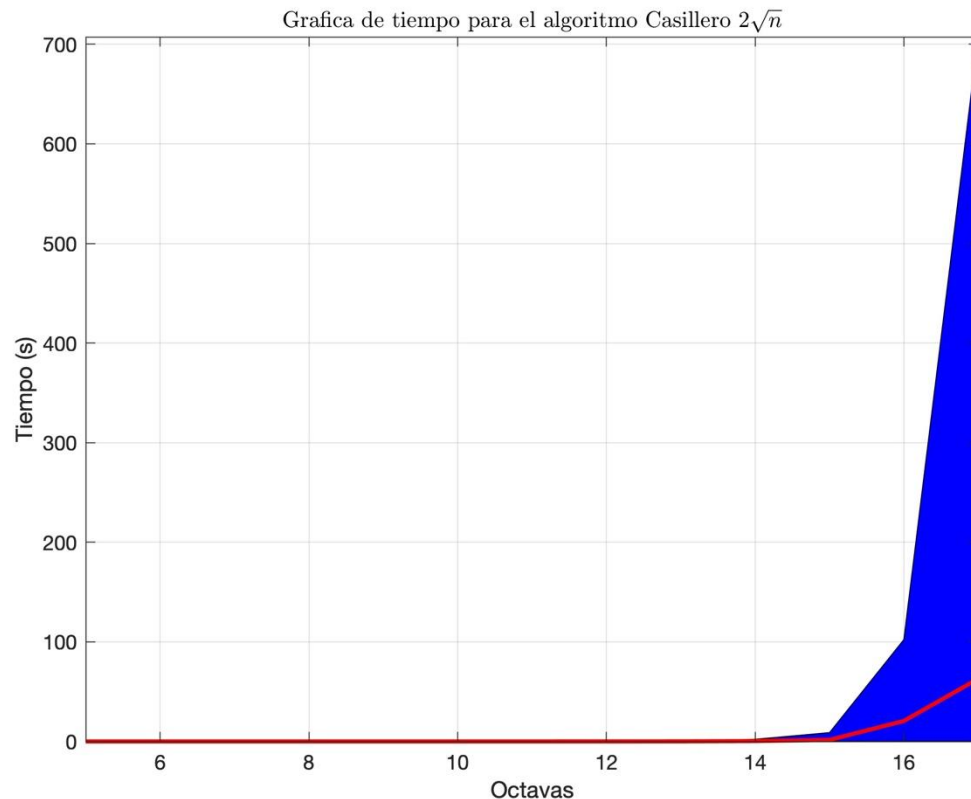
Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra.

Considerando $2\sqrt{n}$ casillas

Características del sistema de computo:

Procesador: Intel Core i5 (4 núcleos)

1.4 GHz

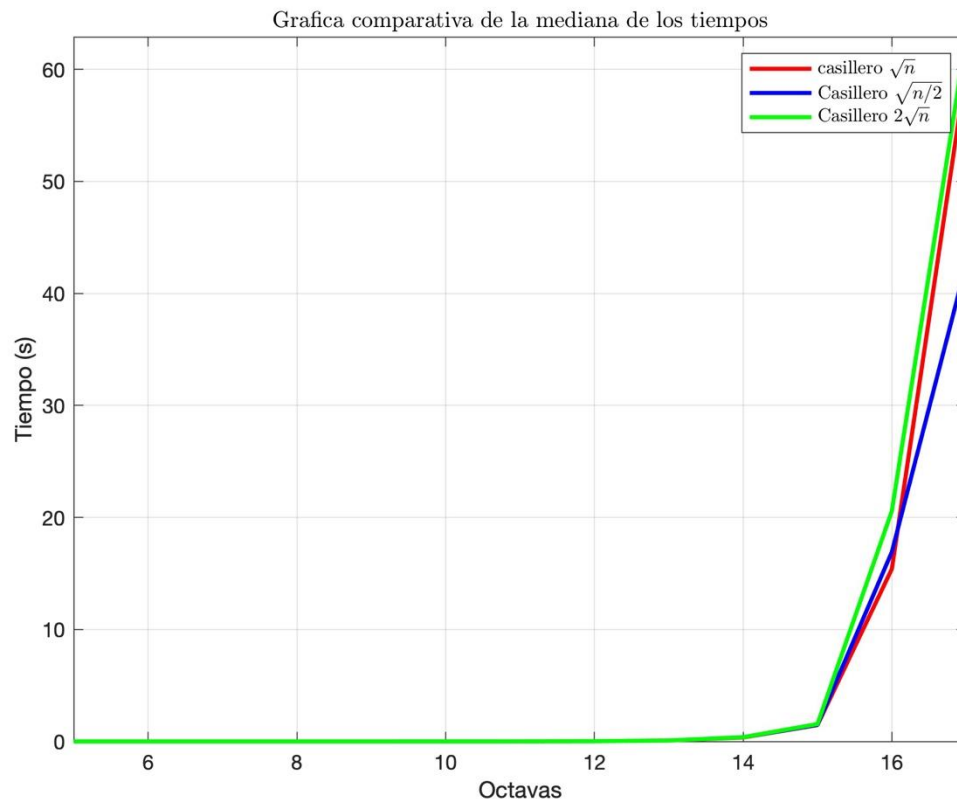
Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra para cada algoritmo y los mismos vectores son usados en cada repetición.

Características del sistema de computo:

Procesador: Intel Core i5 (4 núcleos)
1.4 GHz

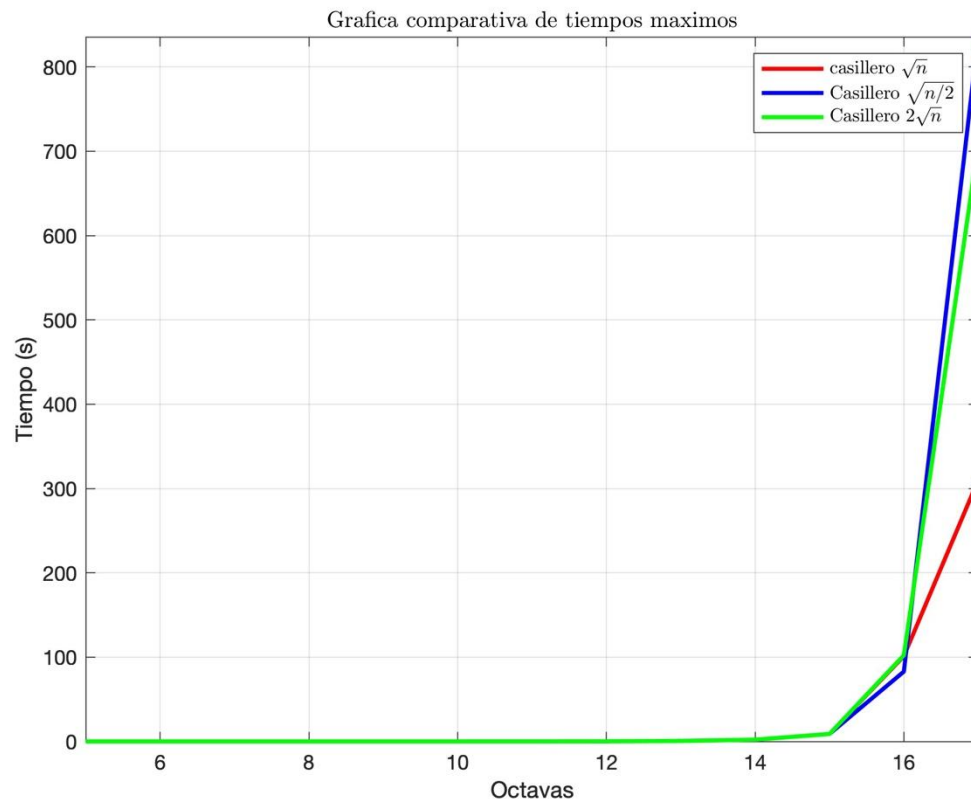
Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

Xcode 12.5.1

Algoritmo de casilleros (Buket sort)

El algoritmo de ordenamiento por casilleros distribuye todos los elementos a ordenar entre un número finito de casilleros. Cada casillero solo puede contener los elementos que cumplan unas determinadas condiciones. Las condiciones corresponden a intervalos de números. Las condiciones deben ser excluyentes entre sí, para evitar que un elemento pueda ser clasificado en dos casilleros distintos. Después cada uno de esos casilleros se ordena individualmente con otro algoritmo de ordenación (que podría ser distinto según el casillero), o se aplica recursivamente este algoritmo para obtener casilleros con menos elementos.



Número de muestras está representado en octavas, desde la quinta octava ($2^5 = 32$) hasta la decimo séptima octava ($2^{17} = 131,072$).

El proceso fue repetido 15 veces para cada número de muestra para cada algoritmo y los mismos vectores son usados en cada repetición.

Características del sistema de computo:

Procesador: Intel Core i5 (4 núcleos)

1.4 GHz

Memoria: 16 GB 2133 MHz LPDDR3

Sistema: macOS Big Sur

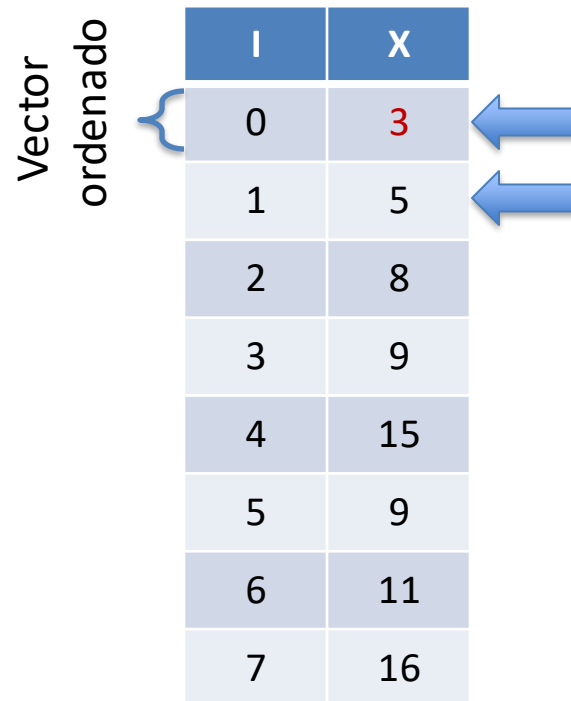
Xcode 12.5.1

Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

Vector ordenado

I	X
0	3
1	5
2	8
3	9
4	15
5	9
6	11
7	16



Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
	4	15	
	5	9	
Vector ordenado	6	11	
	7	16	

Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

		I	X
Vector ordenado	0	3	
	1	5	←
	2	8	←
	3	9	
Vector ordenado	4	15	
	5	9	
	6	11	
	7	16	

Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
	4	11	
Vector ordenado	5	9	←
	6	15	←
	7	16	

Algoritmo burbuja bidireccional (Cocktail sort)

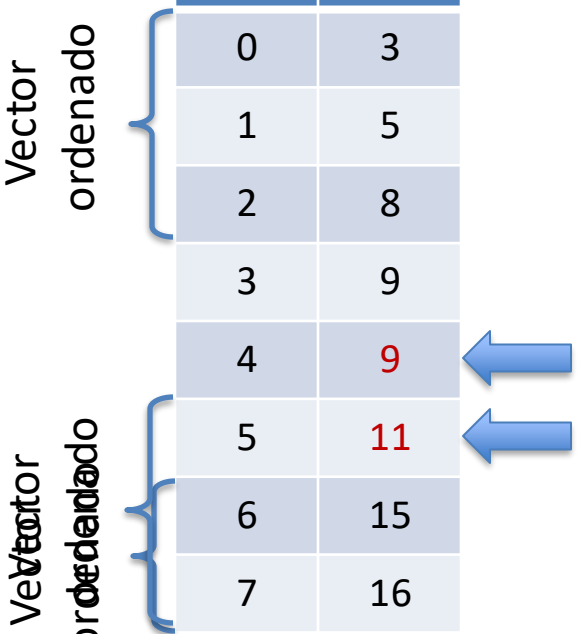
El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	←
	3	9	←
Vector ordenado	4	11	
	5	9	
	6	15	
	7	16	

Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
	3	9	
Vector ordenado	4	9	
	5	11	
	6	15	
	7	16	



Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

		I	X
Vector ordenado	0	3	
	1	5	
	2	8	
Vector ordenado	3	9	←
	4	9	←
Vector ordenado	5	11	
	6	15	
	7	16	

Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.

1. **Estabilidad:** Sí
2. **En su lugar (memoria) :** Sí
3. **Complejidad del mejor caso:** $O(n)$
4. **Complejidad del caso promedio:** $O(n^2)$
5. **Complejidad del peor caso:** $O(n^2)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

Algoritmo burbuja bidireccional (Cocktail sort)

El ordenamiento de burbuja bidireccional (cocktail sort en inglés) es un algoritmo de ordenamiento que surge como una mejora del algoritmo ordenamiento de burbuja. La manera de trabajar de este algoritmo es ir ordenando al mismo tiempo por los dos extremos del vector. De manera que tras la primera iteración, tanto el menor como el mayor elemento estarán en sus posiciones finales. De esta manera se reduce el número de comparaciones aunque la complejidad del algoritmo es la misma.



Algoritmo de conteo (Counting sort)

El algoritmo de conteo utiliza un histograma de los datos entrada para poder ordenar el vector.

I	X
0	5
1	3
2	8
3	9
4	15
5	9
6	11
7	16



I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16



		1		1			1	2		1				1	1			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Algoritmo de conteo (Counting sort)

El algoritmo de conteo utiliza un histograma de los datos entrada para poder ordenar el vector.

1. Estabilidad: Sí
2. En su lugar (memoria) : No
3. Complejidad del mejor caso: $O(n)$
4. Complejidad del caso promedio: $O(n)$
5. Complejidad del peor caso: $O(n)$

I	X
0	3
1	5
2	8
3	9
4	9
5	11
6	15
7	16

Algoritmo de conteo (Counting sort)

El algoritmo de conteo utiliza un histograma de los datos entrada para poder ordenar el vector.



Algoritmo de conteo (Counting sort)

El algoritmo de conteo utiliza un histograma de los datos entrada para poder ordenar el vector.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define N 10000
#define Nh 150

int main(int argc, char *argv[])
{
    int edad[N], i, j, k, n, max, min, h[Nh];
    float ep, med;
    do{
        printf("Ingrese el numero de personas: ");
        scanf("%d", &n);
    }while(n<1 | n>N);
    do{
        printf("Ingrese la edad maxima: ");
        scanf("%d", &max);
        printf("Ingrese la edad minima: ");
        scanf("%d", &min);
        if(max<min)
        {
            max^=min;
            min=max^min;
            max^=min;
        }
    }while((max-min)>Nh);
    printf("[max = %d\tdown = %d]\n", max, min);
    srand(time(NULL));

    for(i=0; i<n; i++)
    {
        edad[i]=rand()%(max-min)+min;
    }
    for(i=0; i<n; i++)
        printf("edad[%d] = %d\n", i+1, edad[i]);
    for(i=1, ep=edad[0]; i<n; i++)
        ep+=edad[i];
    ep/=n;
    printf("Edad promedio: %f\n", ep);
    for(i=0; i<Nh; i++)
        h[i]=0;
    for(i=0; i<n; i++)
        h[edad[i]-min]++;

    i=0;
    j=0;
    while(i<n)
    {
        while(!h[j])
            j++;
        for(k=0; k<h[j]; k++)
        {
            edad[i]=j+min;
            i++;
        }
        j++;
    }
    printf("Vector ordenado.\n");
    for(i=0; i<n; i++)
        printf("edad[%d] = %d\n", i+1, edad[i]);
    med = (n%2?edad[n/2]:(edad[n/2]+edad[n/2-1])/2.0);
    printf("Mediana = %f\n", med);
    return 0;
}
```


Arreglos en dos dimensiones

El arreglo es un tipo de dato derivado, representa una colección ordenada de valores (elementos) de otro tipo. La mayoría de arreglos en C tiene un numero fijo de elementos, todos de un mismo tipo. Esta representación almacena los elementos de forma continua en la memoria sin dejar espacios. C permite los arreglos multidimensionales cuyos elementos pueden ser otros arreglos, arreglos a punteros u otras estructuras derivadas. C soporta los arreglos dinámicos cuyo tamaño es determinado en tiempo de ejecución. El estándar C99 y posteriores, soportan la longitud variable de arreglos (VLAs, por sus siglas en inglés Variable Length Arrays)

Declaración de arreglos bidimensionales:

tipo identificador[# dimensión 1][# dimensión 2];

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} \\ a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} \end{bmatrix}$$

Ejemplo:

```
#define N 3  
#define M 4
```

```
int A[N][M];
```

$$A = [a_{1,1} \quad a_{1,2} \quad a_{1,3} \quad a_{1,4} \quad a_{2,1} \quad a_{2,2} \quad a_{2,3} \quad a_{2,4} \quad a_{3,1} \quad a_{3,2} \quad a_{3,3} \quad a_{3,4}]$$

Arreglos en dos dimensiones

Captura de arreglos:

```
for(i=0; i<N; i++)  
    for(j=0; j<M; j++)  
    {  
        printf("A[%d][%d] = ", i+1, j+1);  
        scanf("%f", &(A[i][j]));  
    }
```

$$C = A + B = \begin{bmatrix} c_{1,1} & c_{1,2} & c_{1,3} & c_{1,4} \\ c_{2,1} & c_{2,2} & c_{2,3} & c_{2,4} \\ c_{3,1} & c_{3,2} & c_{3,3} & c_{3,4} \end{bmatrix}$$

Impresión de arreglos:

```
for(i=0; i<N; i++)  
    for(j=0; j<M; j++)  
        printf("A[%d][%d] = %f\n", i+1, j+1, A[i][j]);
```

$$c_{1,1} = a_{1,1} + b_{1,1}$$

$$c_{1,2} = a_{1,2} + b_{1,2}$$

$$c_{1,3} = a_{1,3} + b_{1,3}$$

Operaciones escalares:

$$c_{i,j} = a_{i,j} + b_{i,j}$$

```
for(i=0; i<N; i++)  
    for(j=0; j<M; j++)  
        C[i][j] = A[i][j] + B[i][j]
```

Arreglos en dos dimensiones

Multiplicación vectorial

$$C = A \times B = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \times \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,1} & b_{3,2} & b_{3,3} \end{bmatrix} = \begin{bmatrix} c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix}$$

$$c_{1,1} = a_{1,1}b_{1,1} + a_{1,2}b_{2,1} + a_{1,3}b_{3,1}$$

$$c_{2,1} = a_{2,1}b_{1,1} + a_{2,2}b_{2,1} + a_{2,3}b_{3,1}$$

$$c_{3,1} = a_{3,1}b_{1,1} + a_{3,2}b_{2,1} + a_{3,3}b_{3,1}$$

$$c_{1,2} = a_{1,1}b_{1,2} + a_{1,2}b_{2,2} + a_{1,3}b_{3,2}$$

$$c_{2,2} = a_{2,1}b_{1,2} + a_{2,2}b_{2,2} + a_{2,3}b_{3,2}$$

$$c_{3,2} = a_{3,1}b_{1,2} + a_{3,2}b_{2,2} + a_{3,3}b_{3,2}$$

$$c_{i,j} = a_{i,1}b_{1,j} + a_{i,2}b_{2,j} + a_{i,3}b_{3,j}$$

```
for(i=0; i<nA; i++)
    for(j=0; j<mB; j++)
        for(k=0; k<mA; k++)
            C[i][j]+=(A[i][k]*B[k][j]);
```

Sistemas de ecuaciones algebraicas:

$$f_1(x_1, x_2, \dots, x_n) = 0$$

$$f_2(x_1, x_2, \dots, x_n) = 0$$

$$\vdots$$

$$f_n(x_1, x_2, \dots, x_n) = 0$$

Pueden ser lineales o no, en este caso consideramos ecuaciones algebraicas lineales que son de forma general:

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n$$

donde las a son coeficientes constantes, las b son constantes y n es el número de ecuaciones.

Representación de sistemas de ecuaciones algebraicas mediante matrices:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$[A]\{x\} = \{b\} \qquad [A] = [\{a_1\} \quad \{a_2\} \quad \dots \quad \{a_n\}]$$

$$[A] = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

Selección de renglón

$$\{R_i\} = [a_{i1} \quad a_{i2} \quad \cdots \quad a_{in}]$$

Selección de columna

$$\{C_j\} = \begin{bmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{nj} \end{bmatrix}$$

Supongamos un sistemas de ecuaciones:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Representado de forma reducida:

$$\left[\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \end{array} \right]$$

El objetivo de la eliminación de Gauss es llegar a un matriz triangular superior mediante un eliminación hacia delante:

$$\left[\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ 0 & a'_{22} & a'_{23} & b'_2 \\ 0 & 0 & a''_{33} & b''_3 \end{array} \right]$$

La solución a este sistema esta dado por la sustitución hacia atrás:

$$\begin{aligned} x_3 &= \frac{b''_3}{a''_{33}} \\ x_2 &= \frac{b'_2 - a'_{23}x_3}{a'_{22}} \\ x_1 &= \frac{b_1 - a_{13}x_3 - a_{12}x_2}{a_{11}} \end{aligned}$$

Inicialmente tenemos:

$$\left[\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \end{array} \right]$$

Para eliminar a_{21} y a_{31} de los renglones 2 y 3, se resta a estos el renglón 1 multiplicado por a_{21}/a_{11} y a_{31}/a_{11} , respectivamente, obteniendo:

$$\left[\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ 0 & a'_{22} & a'_{23} & b'_2 \\ 0 & a'_{32} & a'_{33} & b'_3 \end{array} \right]$$

donde:

$$R'_2 = R_2 - \frac{a_{21}R_1}{a_{11}} = \left[0 \quad a_{22} - \frac{a_{21}a_{12}}{a_{11}} \quad a_{23} - \frac{a_{21}a_{13}}{a_{11}} \quad b_2 - \frac{a_{21}b_1}{a_{11}} \right] = [0 \quad a'_{22} \quad a'_{23} \quad b'_2]$$

$$R'_3 = R_3 - \frac{a_{31}R_1}{a_{11}} = \left[0 \quad a_{32} - \frac{a_{31}a_{12}}{a_{11}} \quad a_{33} - \frac{a_{31}a_{13}}{a_{11}} \quad b_3 - \frac{a_{31}b_1}{a_{11}} \right] = [0 \quad a'_{32} \quad a'_{33} \quad b'_3]$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & b_1 \\ 0 & a'_{22} & a'_{23} & b'_2 \\ 0 & a'_{32} & a'_{33} & b'_3 \end{bmatrix}$$

Para eliminar a'_{32} del renglón 3, se resta a este el renglón 2 multiplicado por a'_{32}/a'_{22} , obteniendo:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & b_1 \\ 0 & a'_{22} & a'_{23} & b'_2 \\ 0 & 0 & a''_{33} & b''_3 \end{bmatrix}$$

donde:

$$R''_3 = R'_3 - \frac{a'_{32}R'_2}{a'_{22}} = \begin{bmatrix} 0 & 0 & a'_{33} - \frac{a'_{32}a'_{23}}{a'_{22}} & b'_3 - \frac{a'_{32}b'_2}{a'_{22}} \end{bmatrix} = [0 \quad 0 \quad a''_{33} \quad b''_3]$$

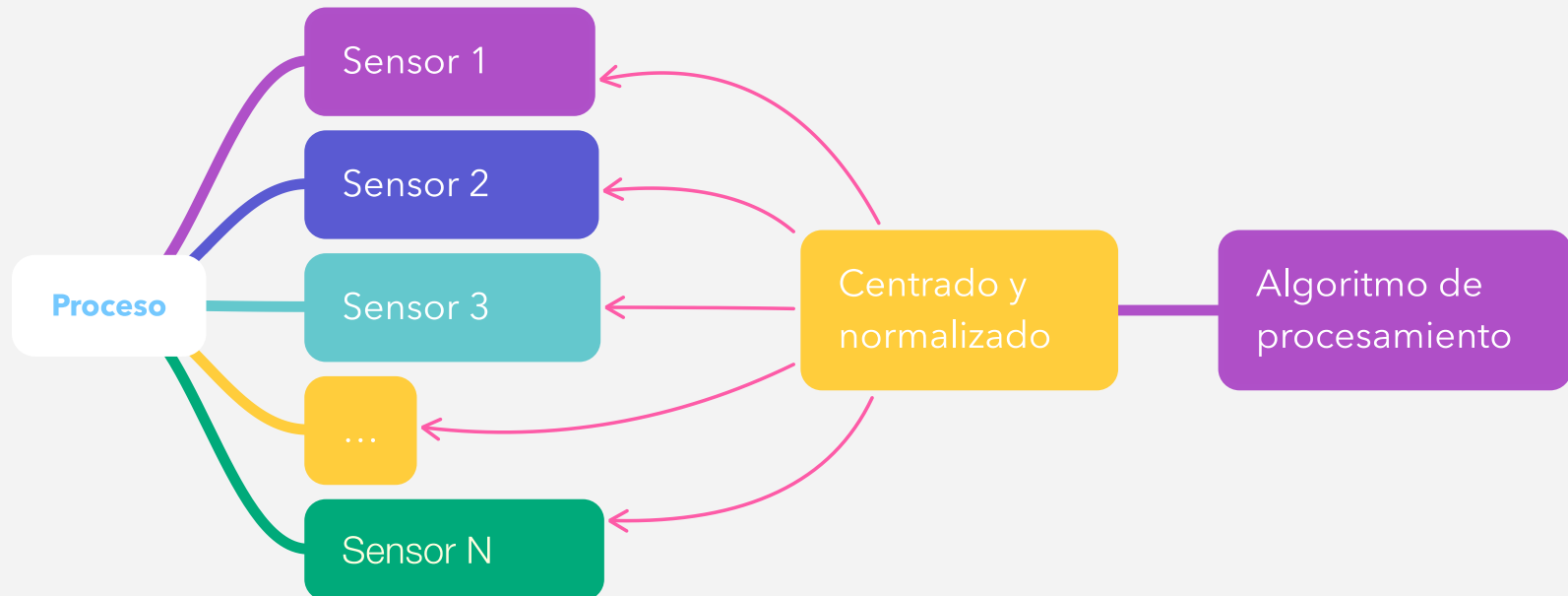
Use la eliminación de Gauss para resolver:

$$\begin{aligned} 3x_1 - 0.1x_2 - 0.2x_3 &= 7.85 \\ 0.1x_1 + 7x_2 - 0.3x_3 &= -19.3 \\ 0.3x_1 - 0.2x_2 + 10x_3 &= 71.4 \end{aligned}$$

La representación matricial sería:

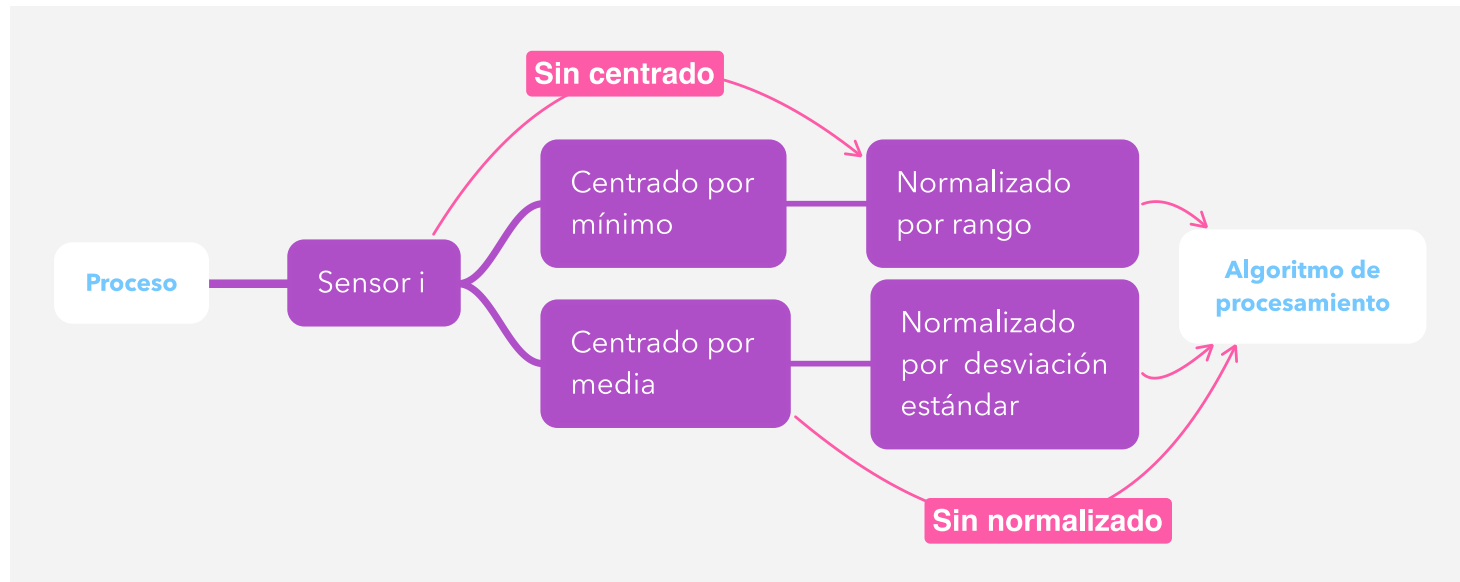
$$\begin{bmatrix} 3 & -0.1 & -0.2 & 7.85 \\ 0.1 & 7 & -0.3 & -19.3 \\ 0.3 & -0.2 & 10 & 71.4 \end{bmatrix}$$

Las soluciones son: $x_1 = 3$, $x_2 = -2.5$ y $x_3 = 7$



- El proceso es medido utilizando diversos sensores, cada sensor puede medir una variable diferente.
- La naturaleza de las señales y los sensores son diferentes y por lo tanto los datos adquiridos tiene diversos comportamientos y rangos.
- Cada sensor es almacenado en una columna diferente y cada muestra de cada sensor es un elemento de la columna.

- El proceso de centrado-normalizado permite estandarizar los datos adquiridos por los sensores, obteniendo la homogeneización en el rango, además de lograr un incremento en la precisión.
- Los dos métodos más utilizados son mediante el rango (máximo y mínimo) y por análisis estadístico (media y desviación estándar)
- El proceso puede hacerse de forma independiente, es decir, el centrado, el normalizado o ambos.



Normalización y centrado

Centrado y normalizado por máximo y mínimo

$$z = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Centrado y normalizado por media y varianza

$$z = \frac{x - \mu}{\sigma}$$

$$\mu = \bar{x} = \frac{\sum x_i}{N}$$

$$\sigma^2 = \frac{\sum (x_i - \mu)^2}{N} = \frac{\sum x_i^2}{N} - \mu^2 = \frac{\sum x_i^2}{N} - \left(\frac{\sum x_i}{N} \right)^2$$

$$\sigma^2 = \frac{\sum (\bar{x} - x_i)^2}{N}$$

$$\sigma^2 = \frac{\sum (\bar{x}^2 - 2\bar{x}x_i + x_i^2)}{N}$$

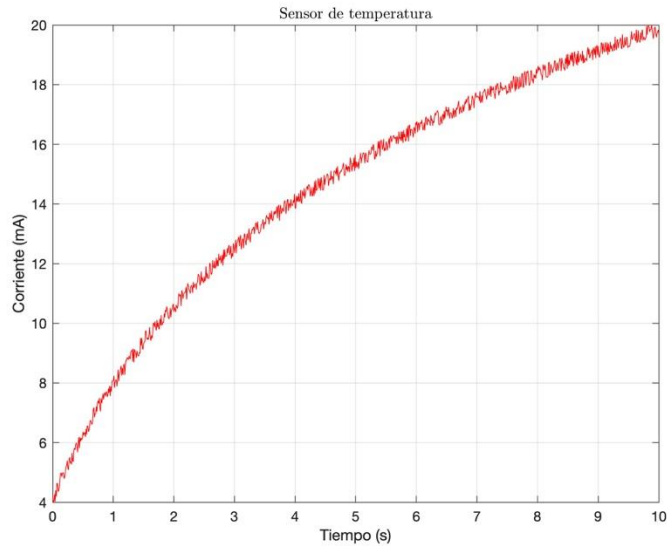
$$\sigma^2 = \frac{N\bar{x}^2 - 2\bar{x} \sum x_i + \sum x_i^2}{N}$$

$$\sigma^2 = \frac{N\bar{x}^2}{N} - \frac{2\bar{x} \sum x_i}{N} + \frac{\sum x_i^2}{N}$$

$$\sigma^2 = \bar{x}^2 - 2\bar{x} \frac{\sum x_i}{N} + \frac{\sum x_i^2}{N}$$

$$\sigma^2 = \bar{x}^2 - 2\bar{x}^2 + \frac{\sum x_i^2}{N}$$

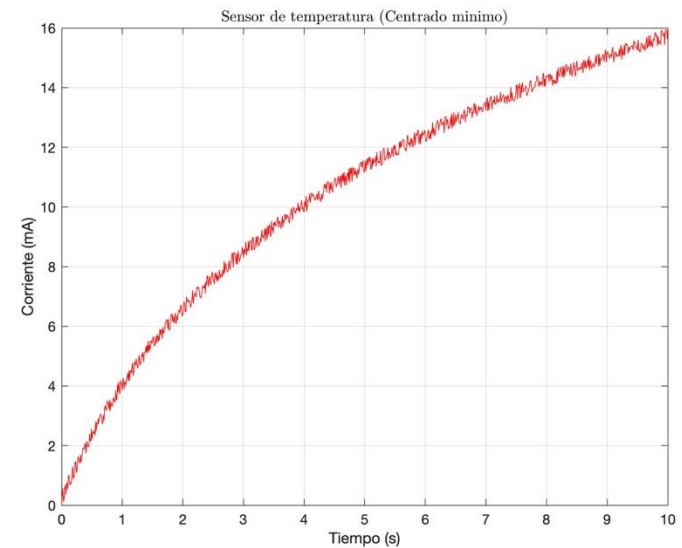
$$\sigma^2 = \frac{\sum x_i^2}{N} - \bar{x}^2 = \frac{\sum x_i^2}{N} - \mu^2$$

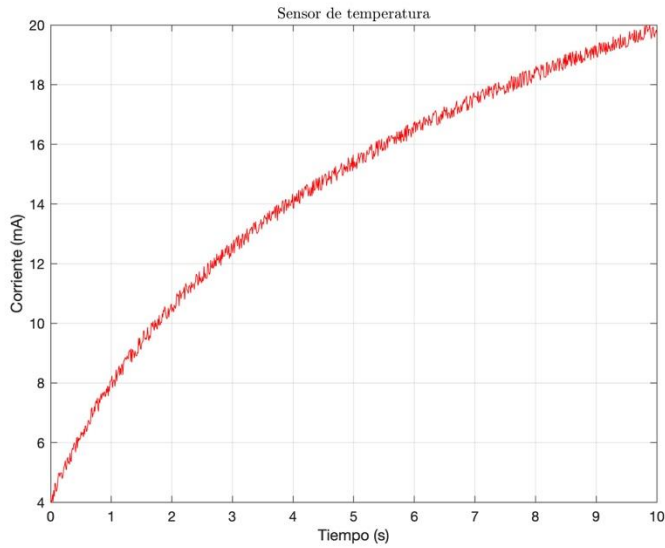


↓

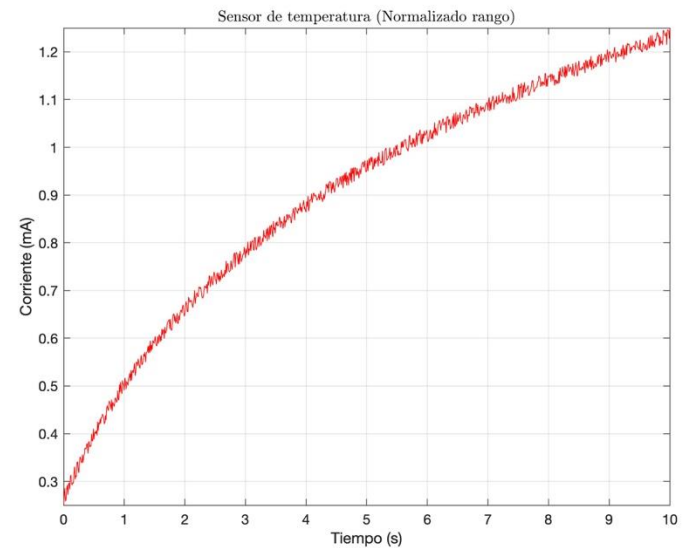
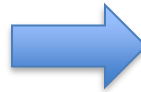
$$z = \frac{x - \min(x)}{1}$$

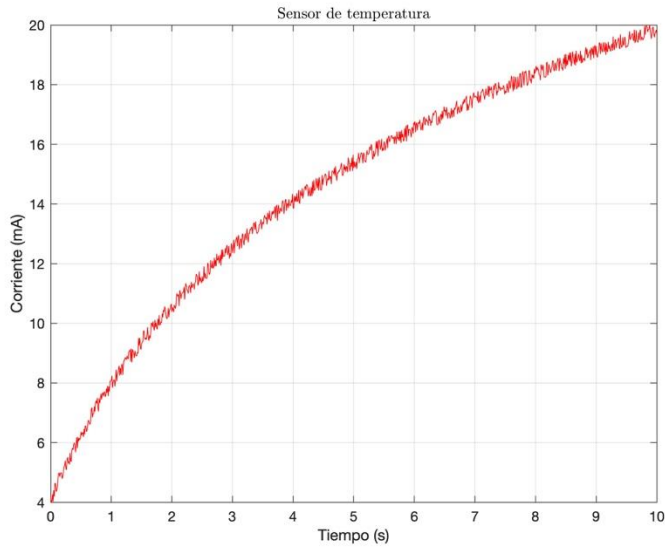
→



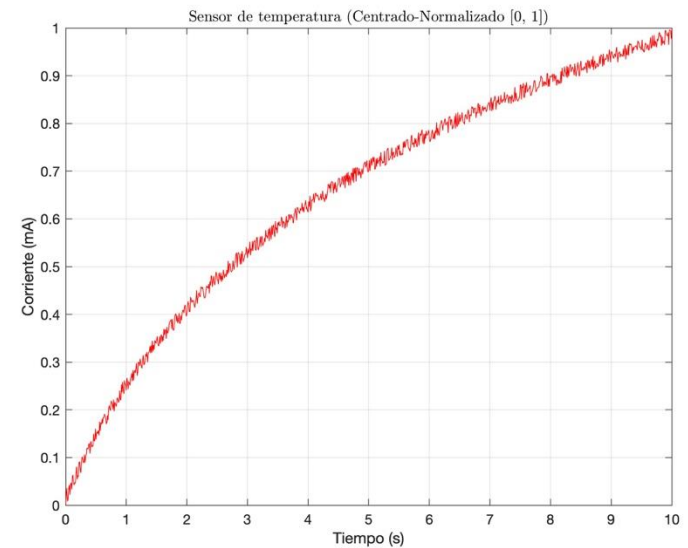
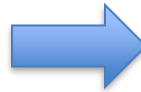


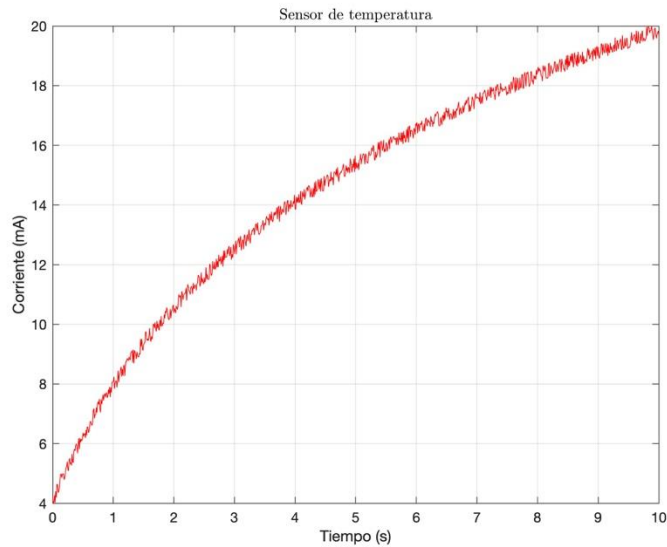
$$z = \frac{x - 0}{\max(x) - \min(x)}$$



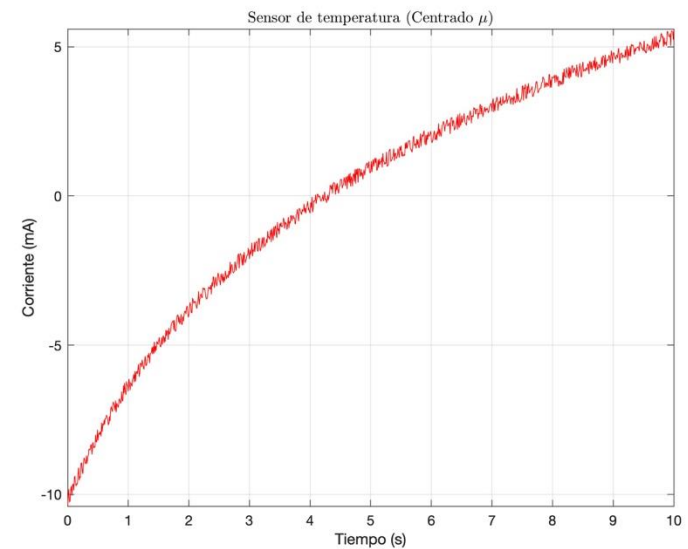
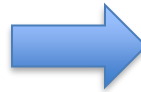


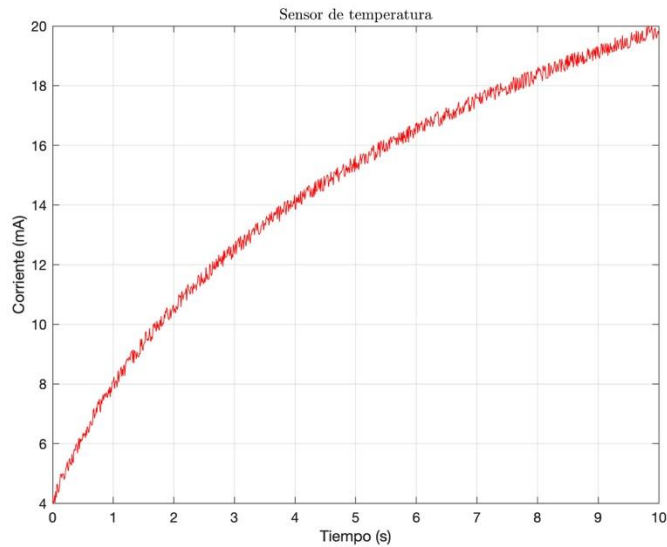
$$z = \frac{x - \min(x)}{\max(x) - \min(x)}$$



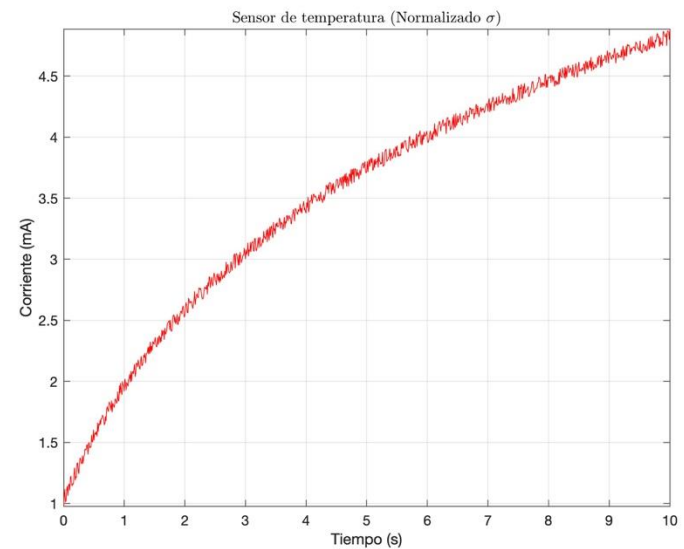
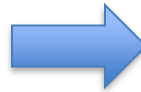


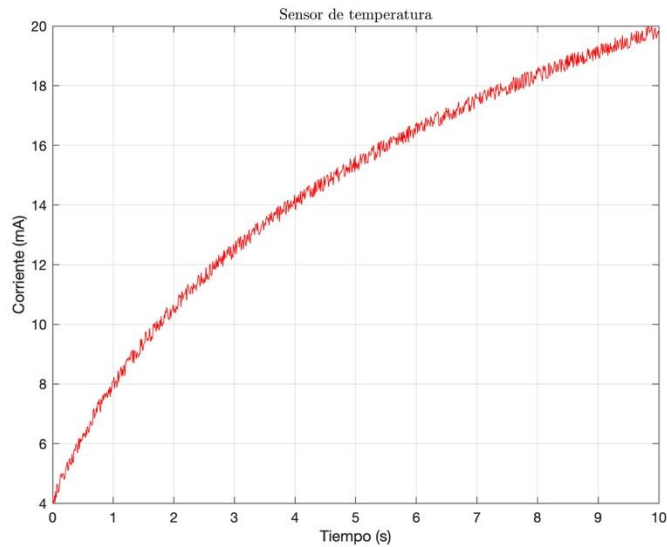
$$z = \frac{x - \mu}{1}$$



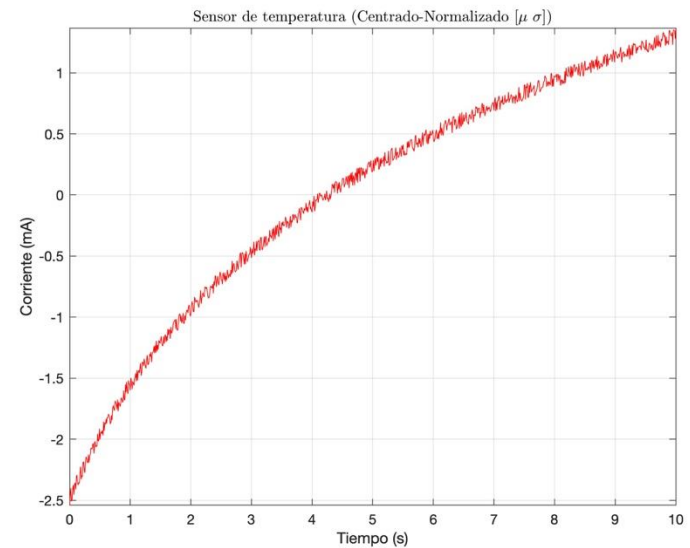
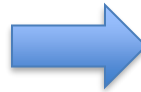


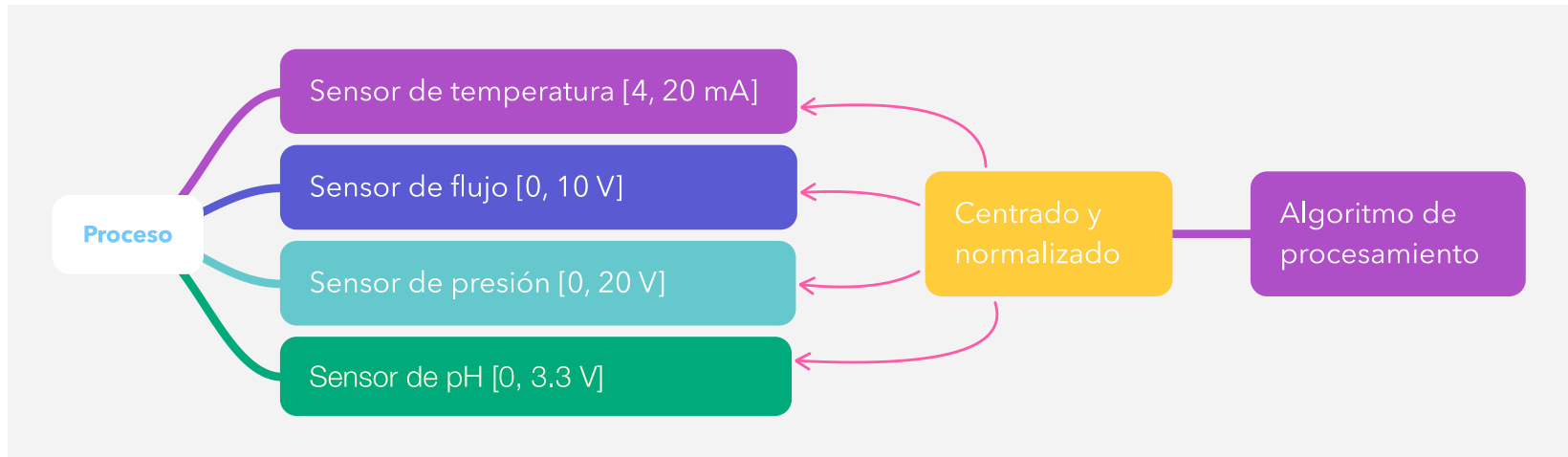
$$z = \frac{x - 0}{\sigma}$$



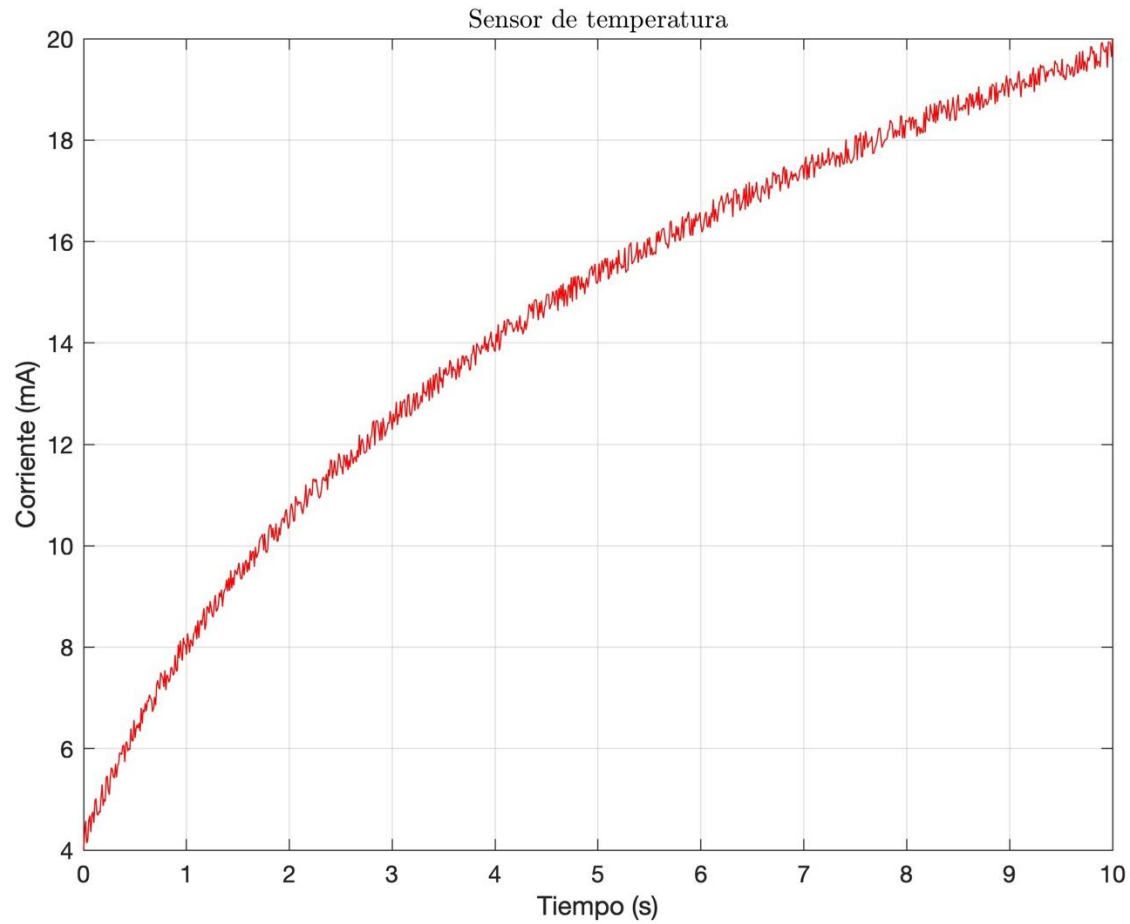


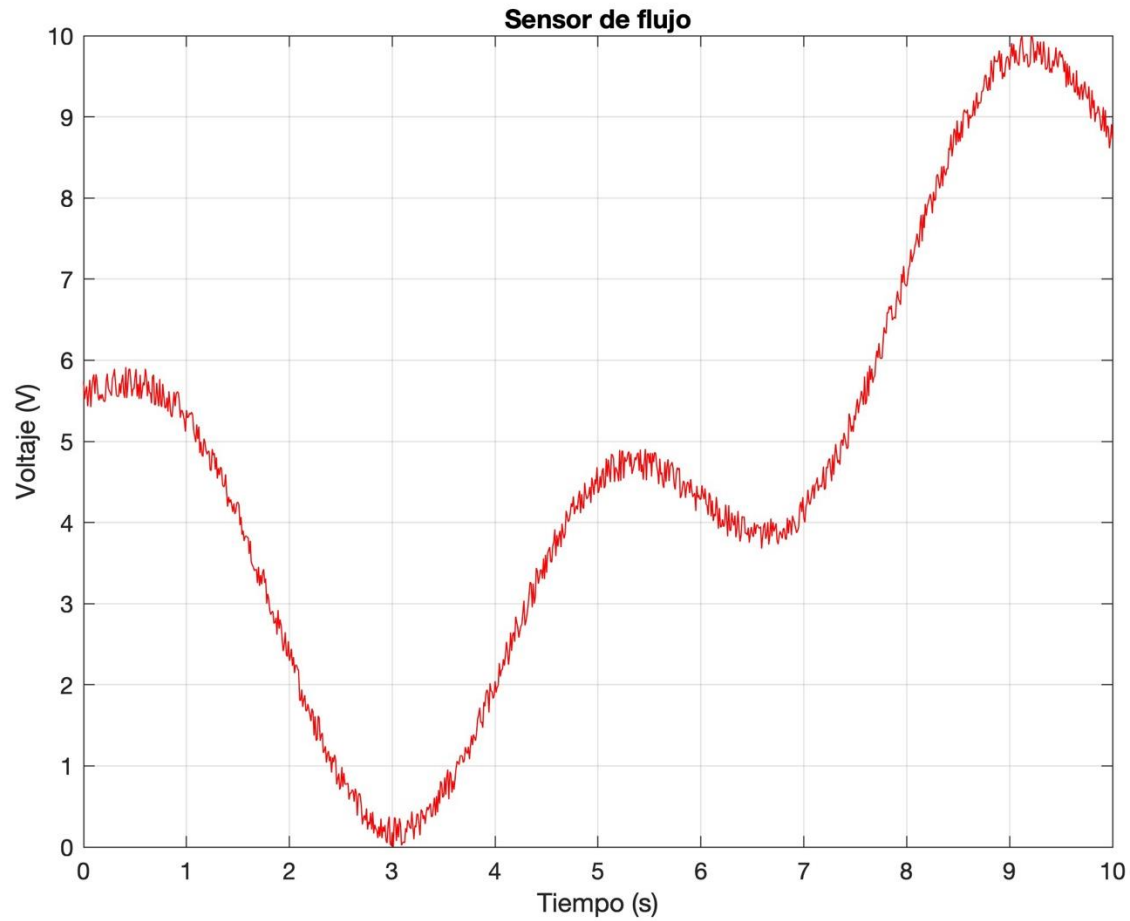
$$z = \frac{x - \mu}{\sigma}$$

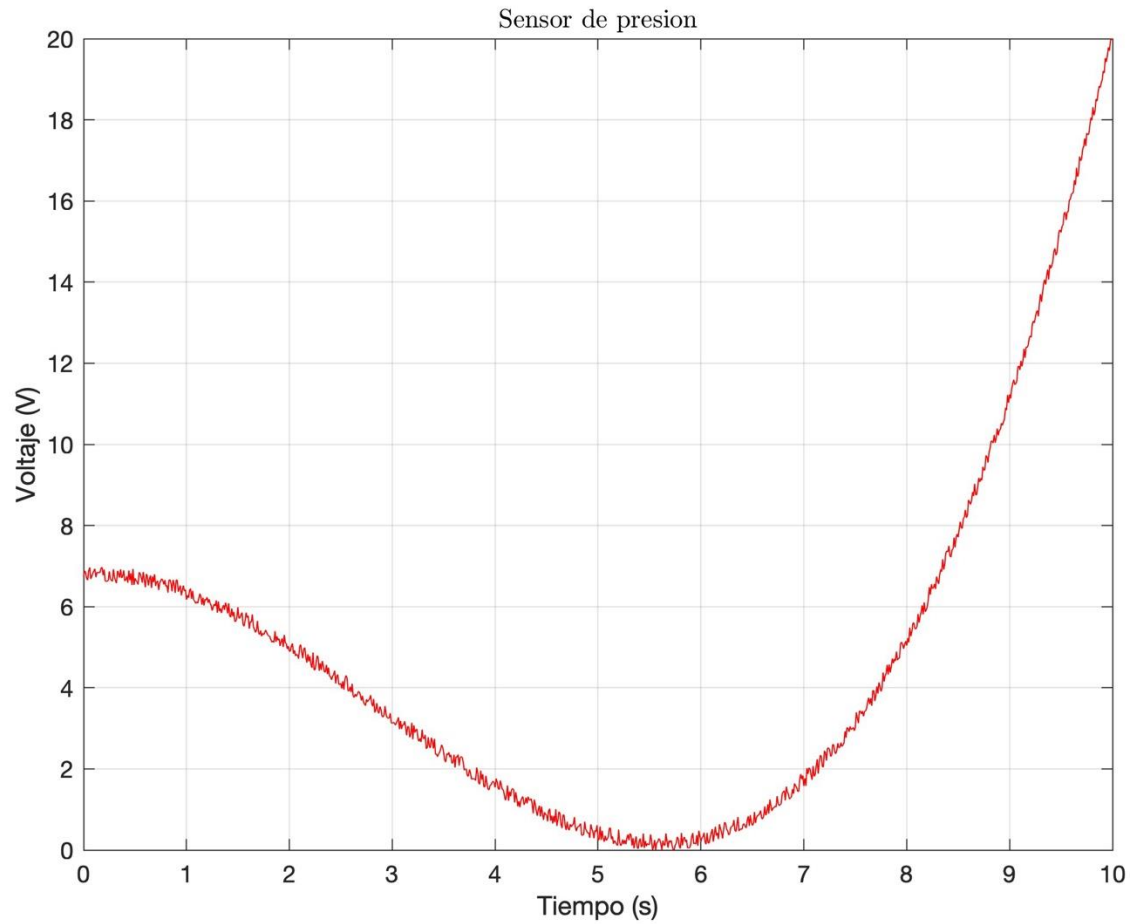


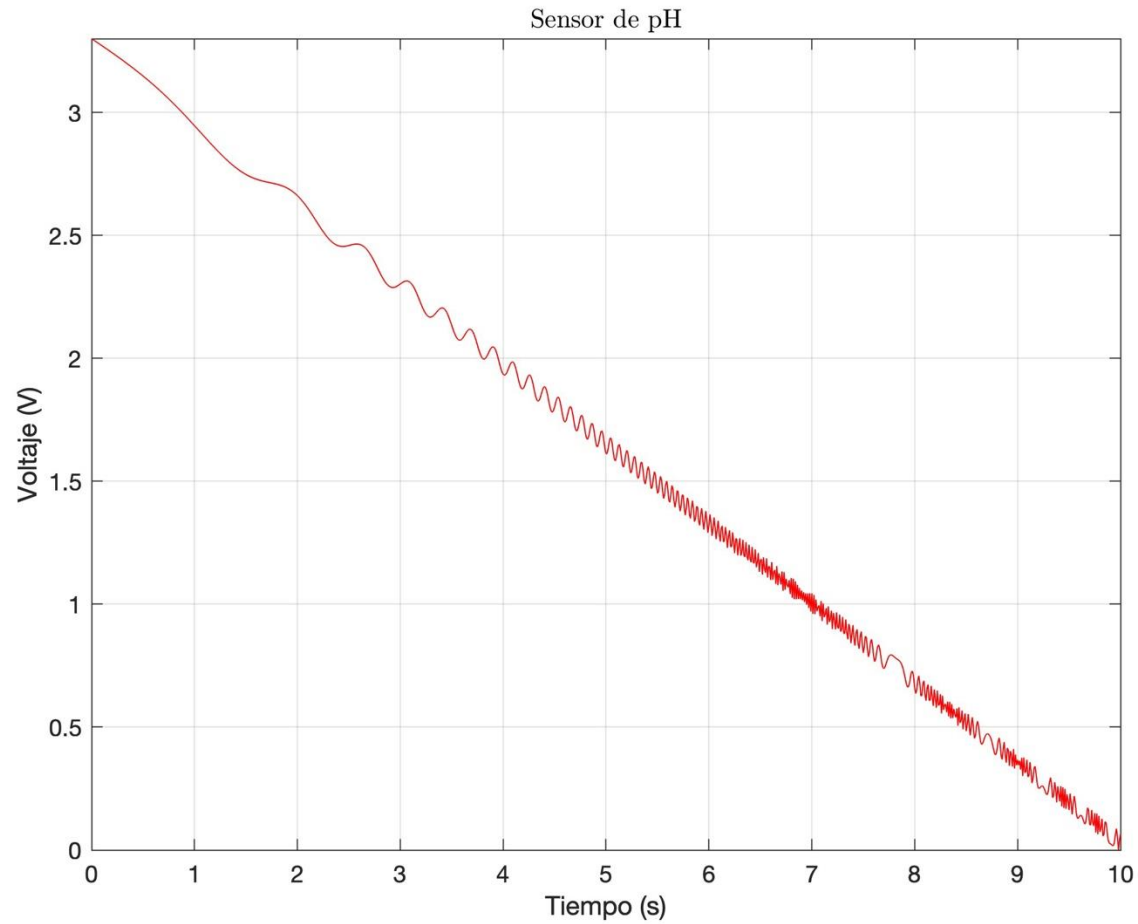


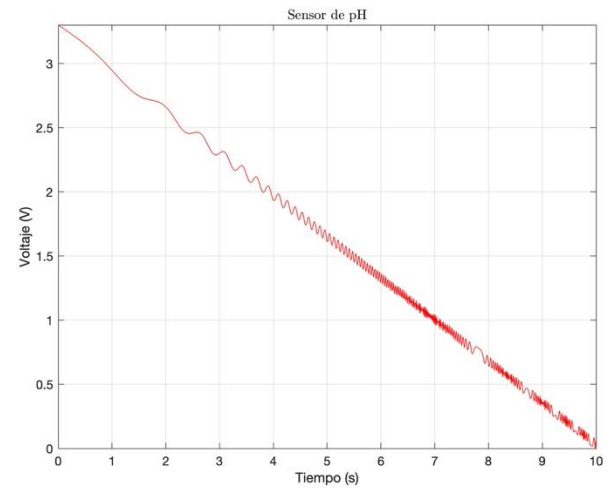
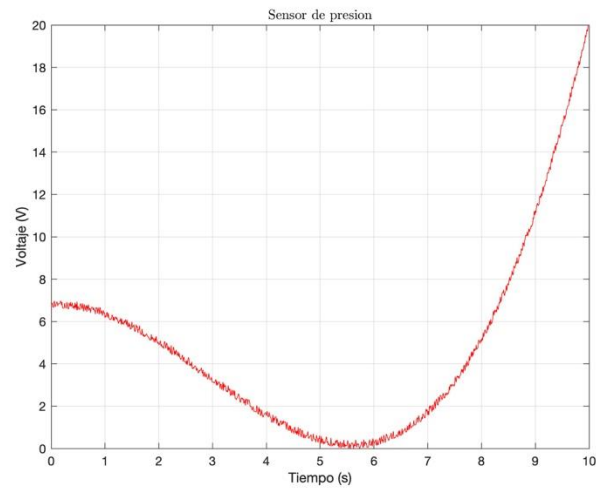
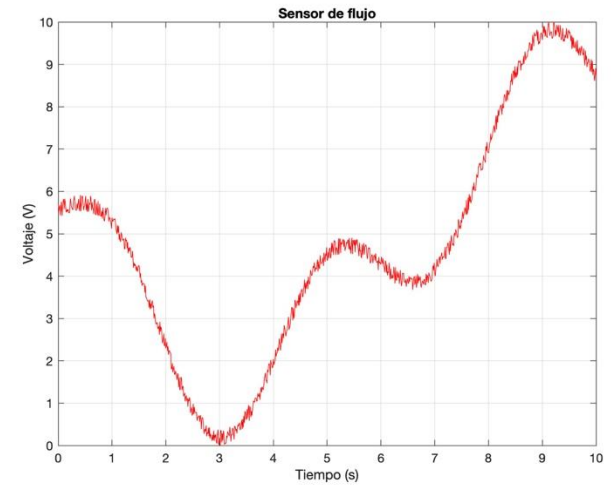
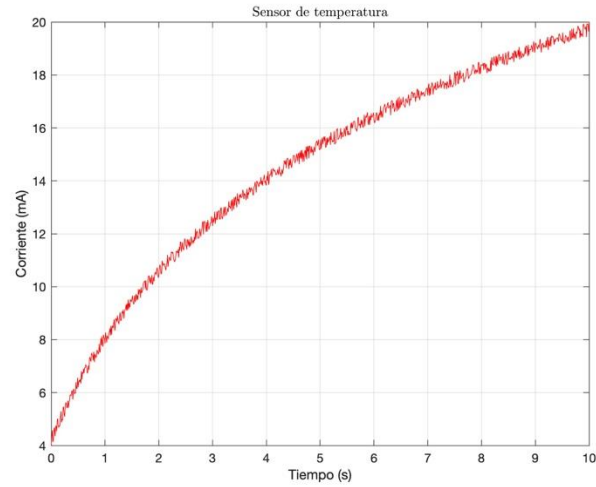
- Supongamos un proceso con cuatro variables, cada variable será una columna de una matriz.
- Cada sensor tiene un rango específico de acuerdo al tiempo de sensor y la naturaleza de la señal adquirida.
 - Temperatura [4-20 mA]
 - Flujo [0-10 V]
 - Presión [0-20 V]
 - pH [0-3.3 V]
- Cada elemento de la columna será una muestra de la señal, considerado un sistema de muestreo síncrono, muestreado durante 10 segundos.

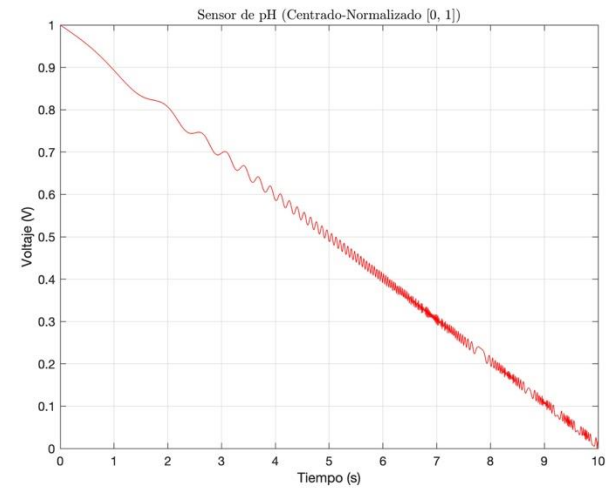
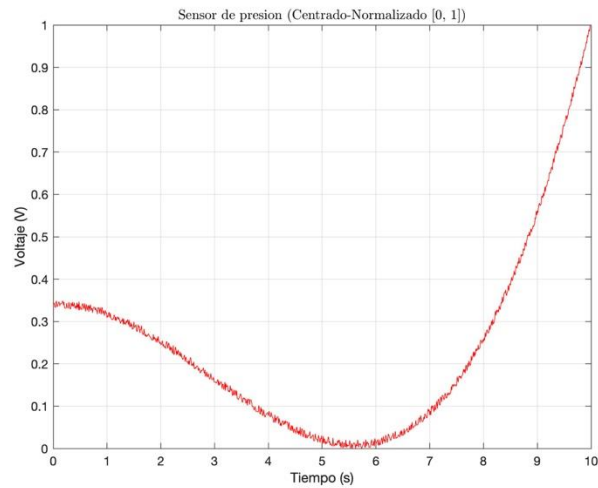
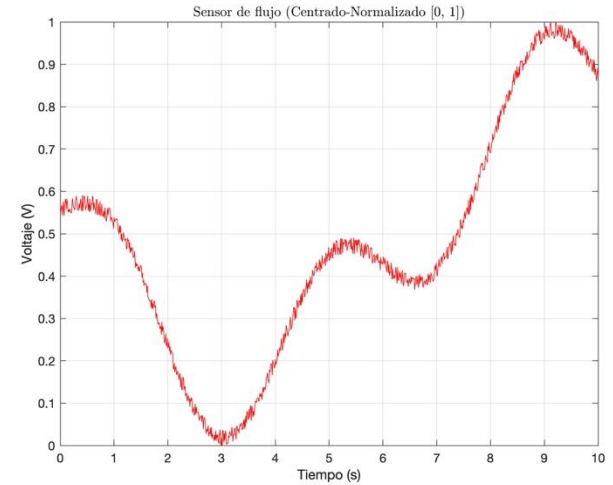
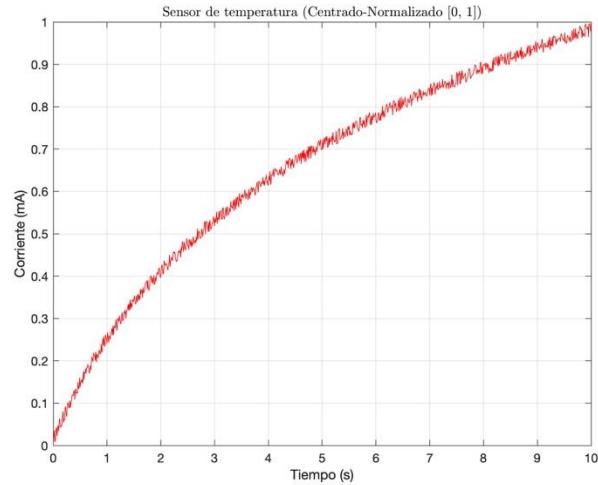


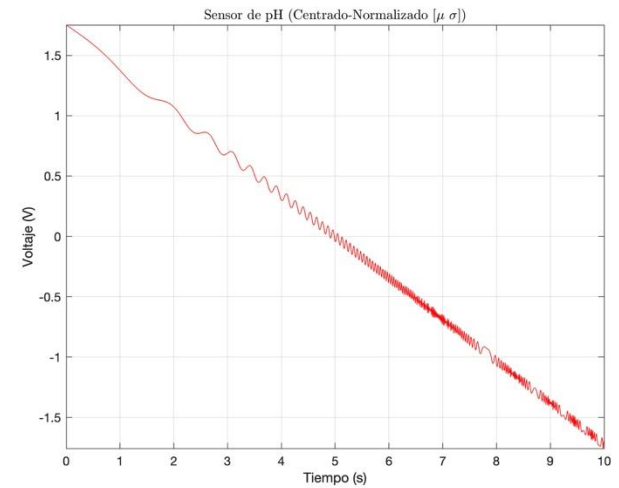
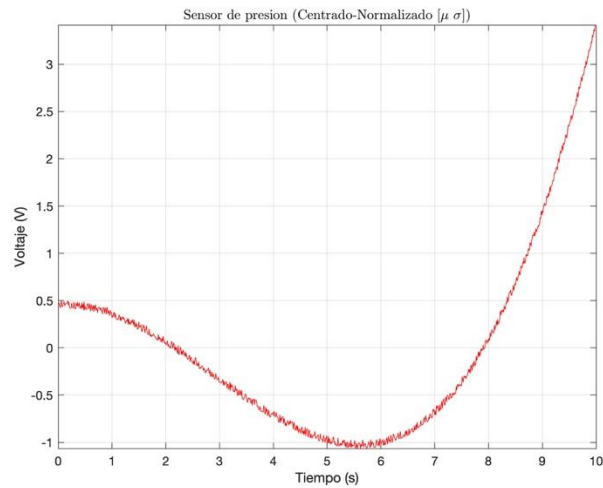
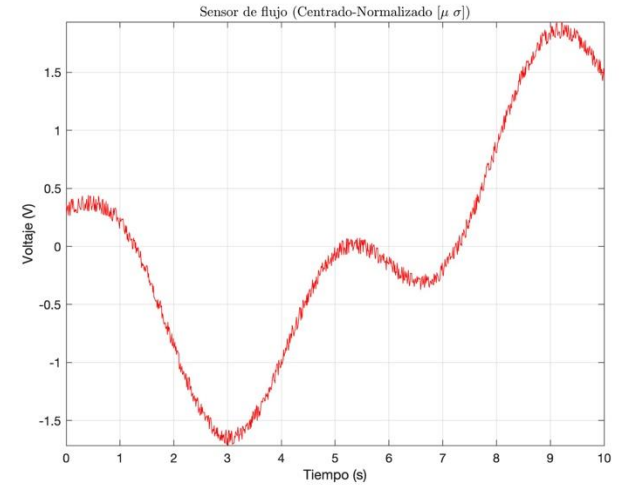
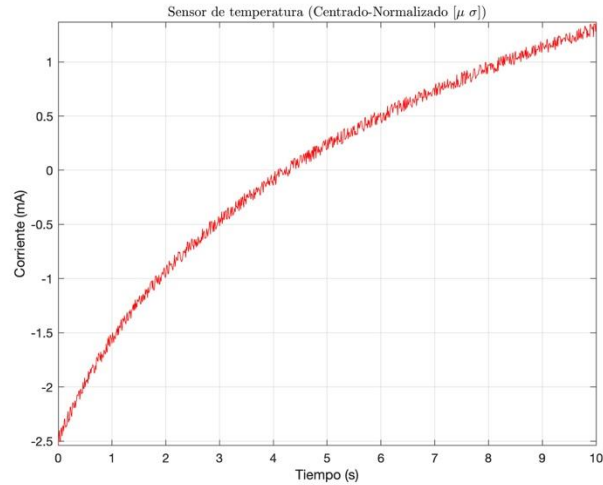












Normalización y centradoBandera = XXXX $b_3b_2b_1b_0$

$$z_i = x_i$$

$$\text{Bandera} = 0000 = 0$$

$$z_i = x_i - \min(x)$$

$$b_0 = 1$$

$$\text{Bandera} = 0001 = 1$$

$$z_i = \frac{x_i}{\max(x) - \min(x)}$$

$$b_1 = 1$$

$$\text{Bandera} = 0010 = 2$$

$$z_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

$$\text{Bandera} = 0011 = 3$$

$$z_i = x_i - \mu$$

$$b_2 = 1$$

$$\text{Bandera} = 0100 = 4$$

$$z_i = \frac{x_i}{\sigma}$$

$$b_3 = 1$$

$$\text{Bandera} = 1000 = 8$$

$$z_i = \frac{x_i - \mu}{\sigma}$$

$$\text{Bandera} = 1100 = 12$$

$$B_0 = 1 \qquad \binom{n}{k} = \frac{n!}{k! \times (n-k)!}$$

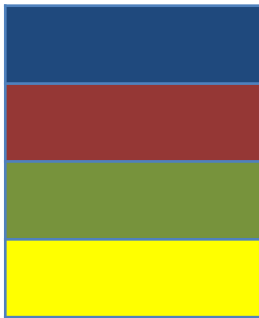
$$B_1 = - \sum_{i=0}^{1-1=0} \binom{1}{i} \frac{B_i}{1+1-i} = - \binom{1}{0} \frac{B_0}{2} = - \frac{1!}{0!1!} \frac{B_0}{2} = -\frac{1}{2}$$

$$B_2 = - \sum_{i=0}^{2-1=1} \binom{2}{i} \frac{B_i}{3-i} = - \left(\binom{2}{0} \frac{B_0}{3} + \binom{2}{1} \frac{B_1}{2} \right) = - \left(\frac{2!}{0!2!} \times \frac{1}{3} + \frac{2!}{1!1!} \times \frac{-1/2}{2} \right)$$

$$B_2 = - \left(\frac{1}{3} - \frac{1}{2} \right) = - \left(\frac{2-3}{6} \right) = \frac{1}{6}$$

$$\binom{n}{k} = \frac{n!}{k! \times (n-k)!} \quad \binom{4}{2} = \frac{4!}{2! \times (4-2)!} = \frac{4!}{2! \times 2!} = \frac{2! \times 3 \times 4}{2! \times 2!} = \frac{3 \times 4}{2} = 3 \times 2 = 6$$

n=4



k=2



UNIVERSIDAD DE GUANAJUATO

